COSC264 · TERM 4

Module 5

Transport Layer Protocols

TCP and UDP

cosc264.up.railway.app

Module 4 — what we covered

- Reliable Data Transfer (RDT) moves data reliably over an unreliable channel.
- RDT uses checksums, acknowledgements (ACKs), sequence numbers, and a timer to detect errors and loss.
- Pipelining uses a window: Go-Back-N uses cumulative ACKs, while Selective Repeat uses individual ACKs.

OUR ROUTE THROUGH THE TRANSPORT LAYER

Module 5 roadmap

01

Transport-layer services

What transport provides to applications: sockets, multiplexing and demultiplexing.

02

UDP and TCP

The two transport protocols: segment formats, connections, and TCP's reliable delivery.

03

TCP flow control

Keeping a fast sender from swamping the receiver.

04

TCP congestion control

Matching the sending rate to what the network can carry.

FROM HOSTS TO APPLICATION PROCESSES

Concept 1 of 4

Transport-layer services

Applications use transport services for process-to-process communication.

END-TO-END ABSTRACTION

Transport layer provides an end-to-end channel between applications

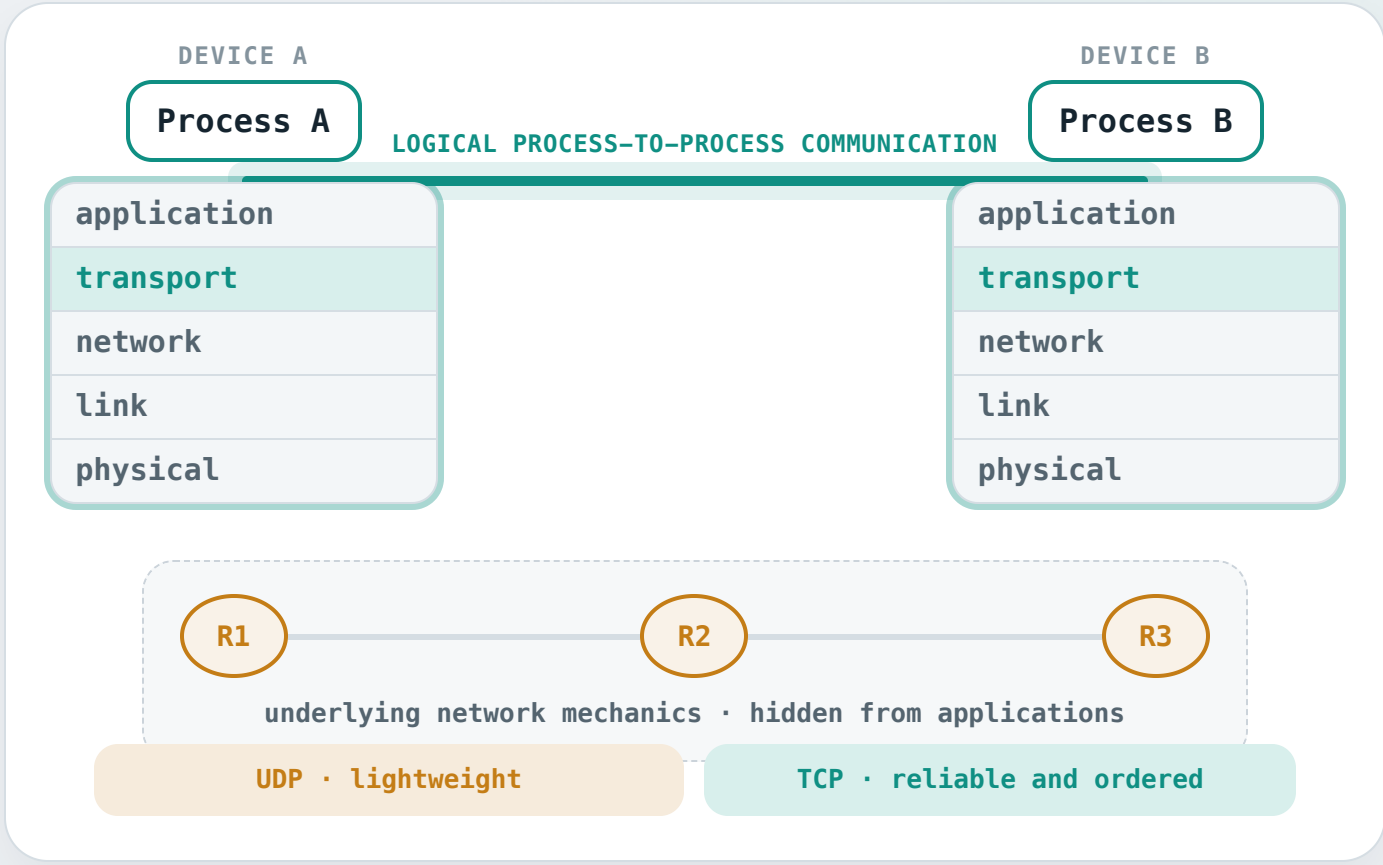
Suppose a network application runs process A and process B on separate devices and needs an end-to-end communication channel.

Transport layer provides that channel. It provides logical end-to-end communication between two processes.

Abstraction. The transport layer abstracts away the network's lower-layer mechanics, keeping them invisible to applications.

Service choice. Applications can choose UDP or TCP for different delivery needs.

UDP TCP



HOW THE ABSTRACTION IS DELIVERED

Mechanisms utilized by transport-layer protocols

To provide this end-to-end abstraction, transport protocols use several mechanisms.

Error detection

A checksum helps detect corruption in a received transport unit.

TCP + UDP

Multiplexing and demultiplexing

Multiplexing and demultiplexing extend host delivery to the correct application process.

TCP + UDP

Reliable data transfer (RDT)

Lost or damaged data is recovered and delivered in order.

TCP

Flow control

Prevents a fast sender from overwhelming its receiver.

TCP

Congestion control

Adapts the sending rate when the network becomes congested.

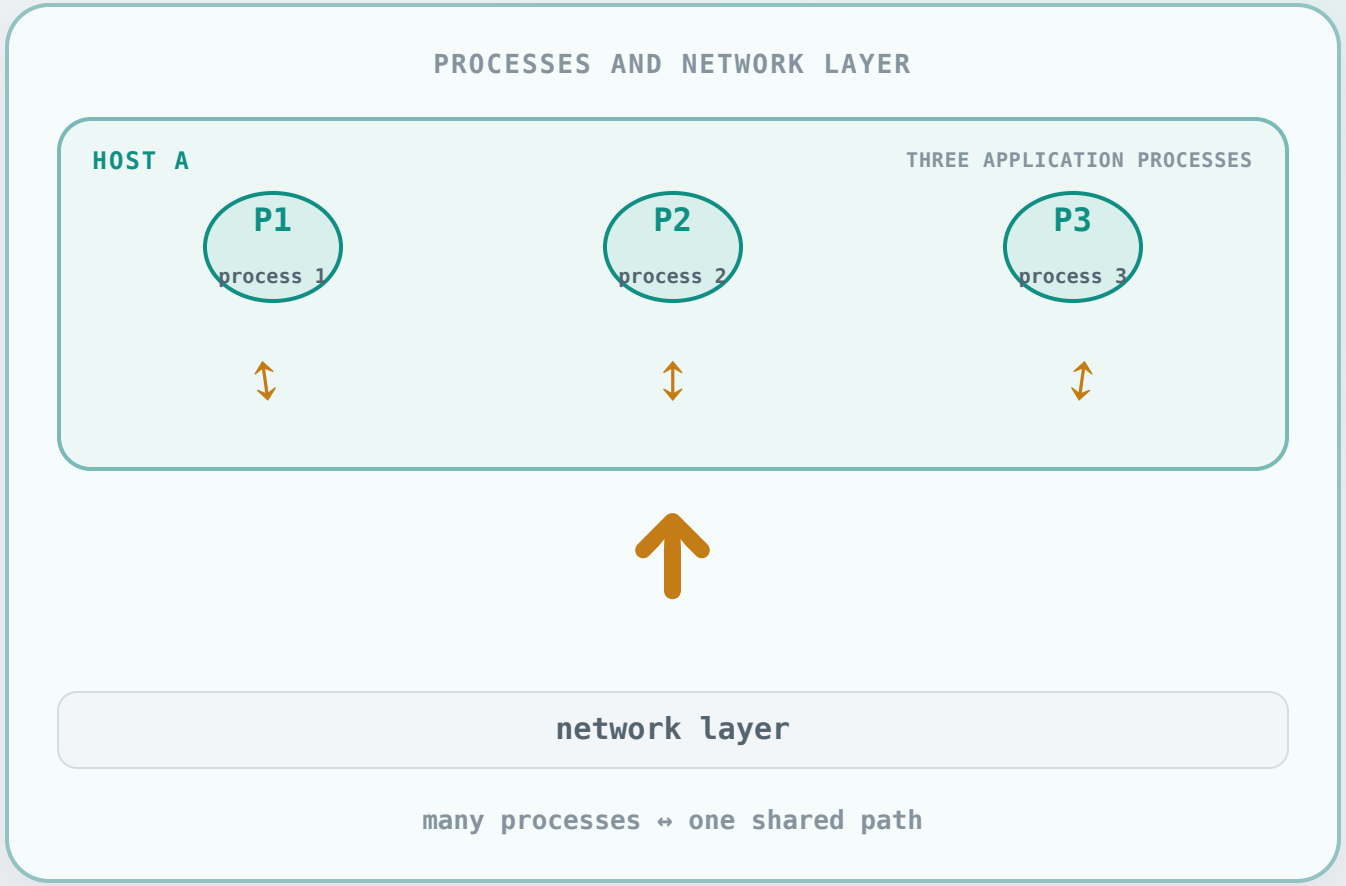
TCP

ONE HOST · MANY APPLICATION PROCESSES

Why multiplexing is needed

A single host can run many network application processes. Each process needs to send and receive traffic through the network.

> The network layer delivers data only to the host, so it cannot identify the destination process.



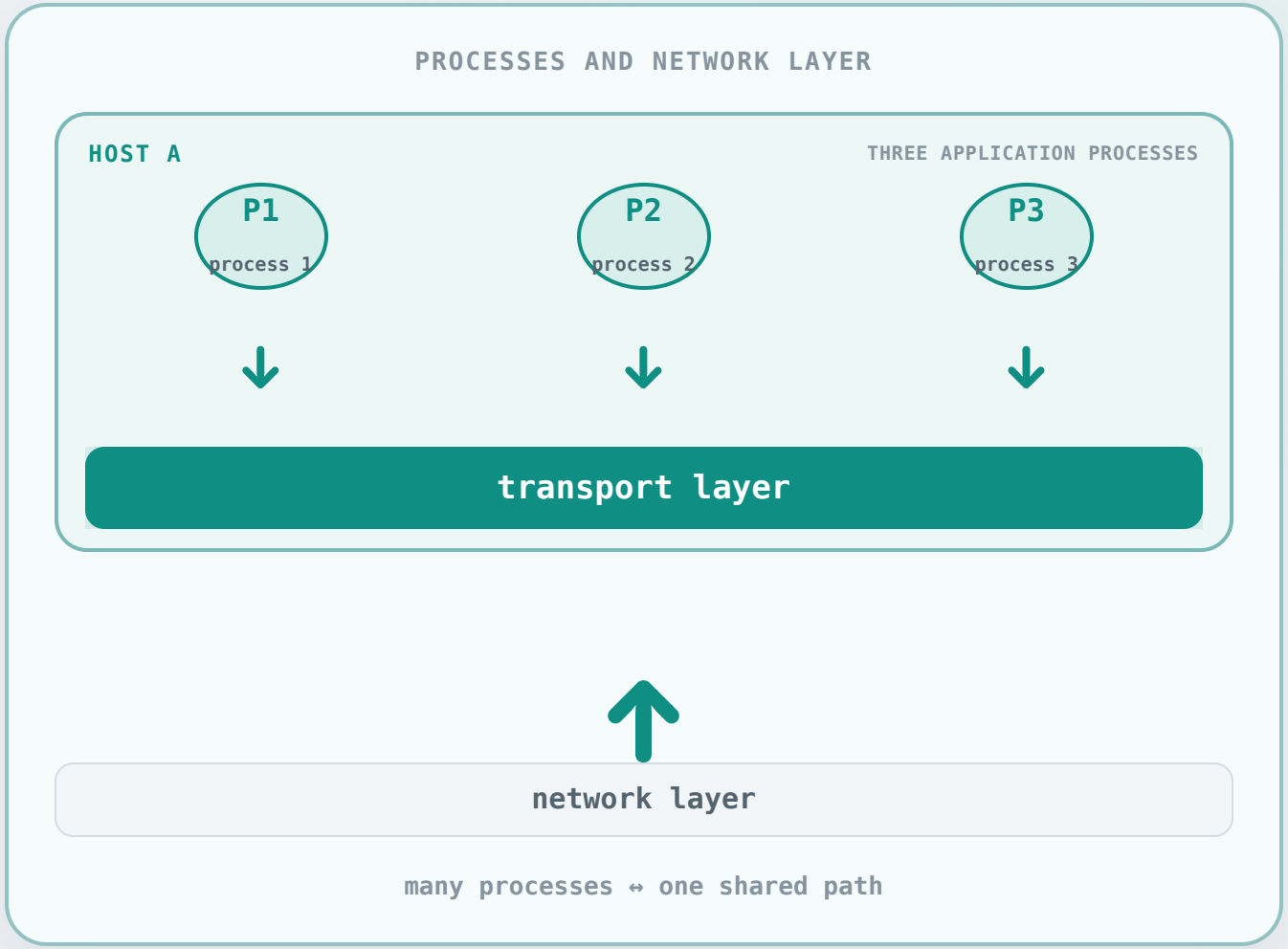
ONE HOST · MANY APPLICATION PROCESSES

Why multiplexing is needed

A single host can run many network application processes. Each process needs to send and receive traffic through the network.

> The network layer delivers data only to the host, so it cannot identify the destination process.

> The transport layer coordinates this traffic in both directions so each process can talk to the network individually. This is known as multiplexing/demultiplexing.

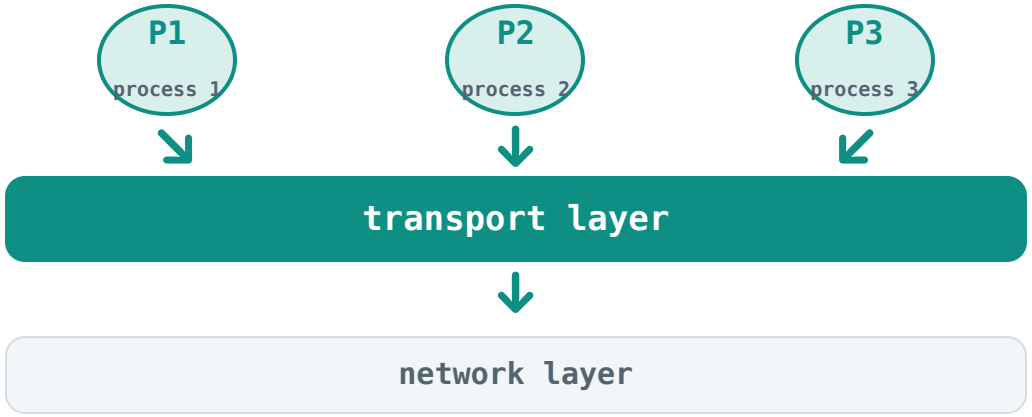


COMBINE OUTGOING DATA · ROUTE INCOMING DATA

Multiplexing sends; demultiplexing delivers

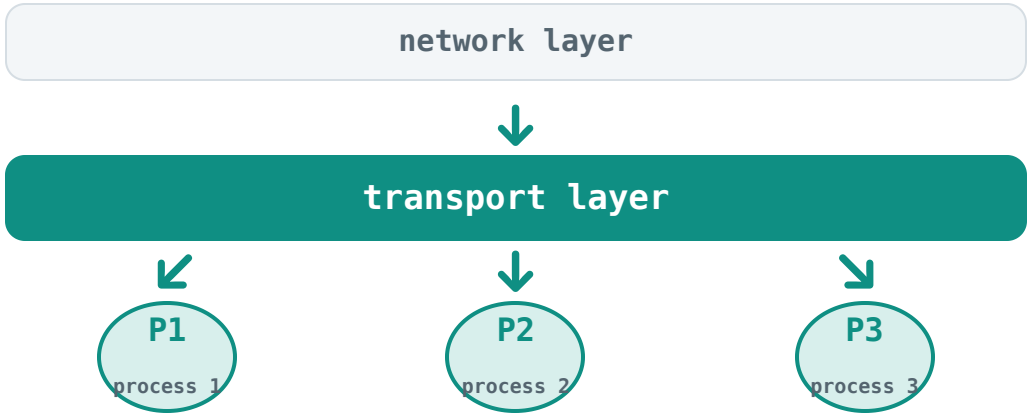
Multiplexing

The sender gathers data from multiple application processes and passes transport units to the network layer.



Demultiplexing

The receiver uses transport-header information to deliver data to the correct application process.

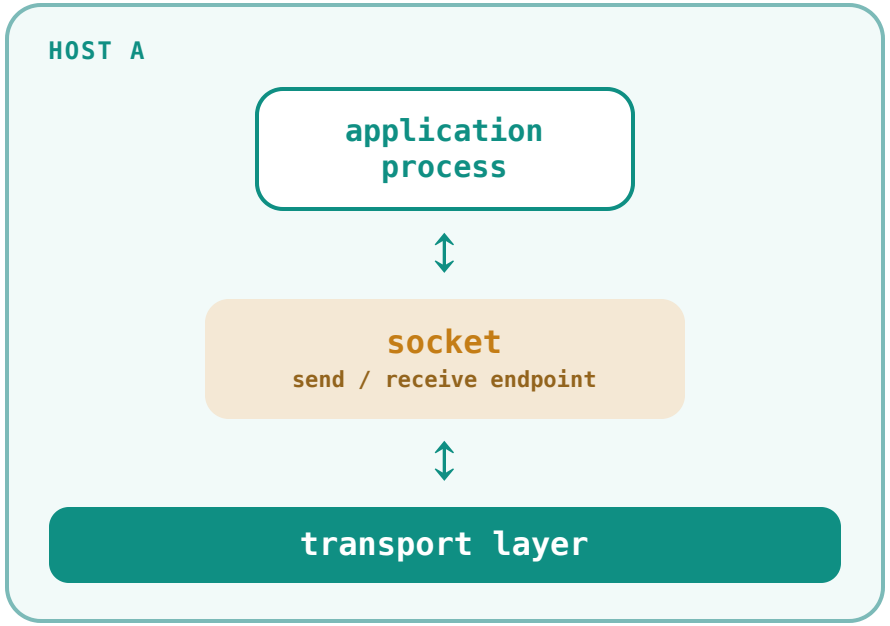


BEFORE DEMULTIPLEXING

Sockets are transport-layer endpoints

A process uses a socket. A socket is the transport-layer endpoint through which an application sends and receives data.

To demultiplex is to find the right socket. For incoming data, the transport layer uses header information to identify the socket that should receive it.

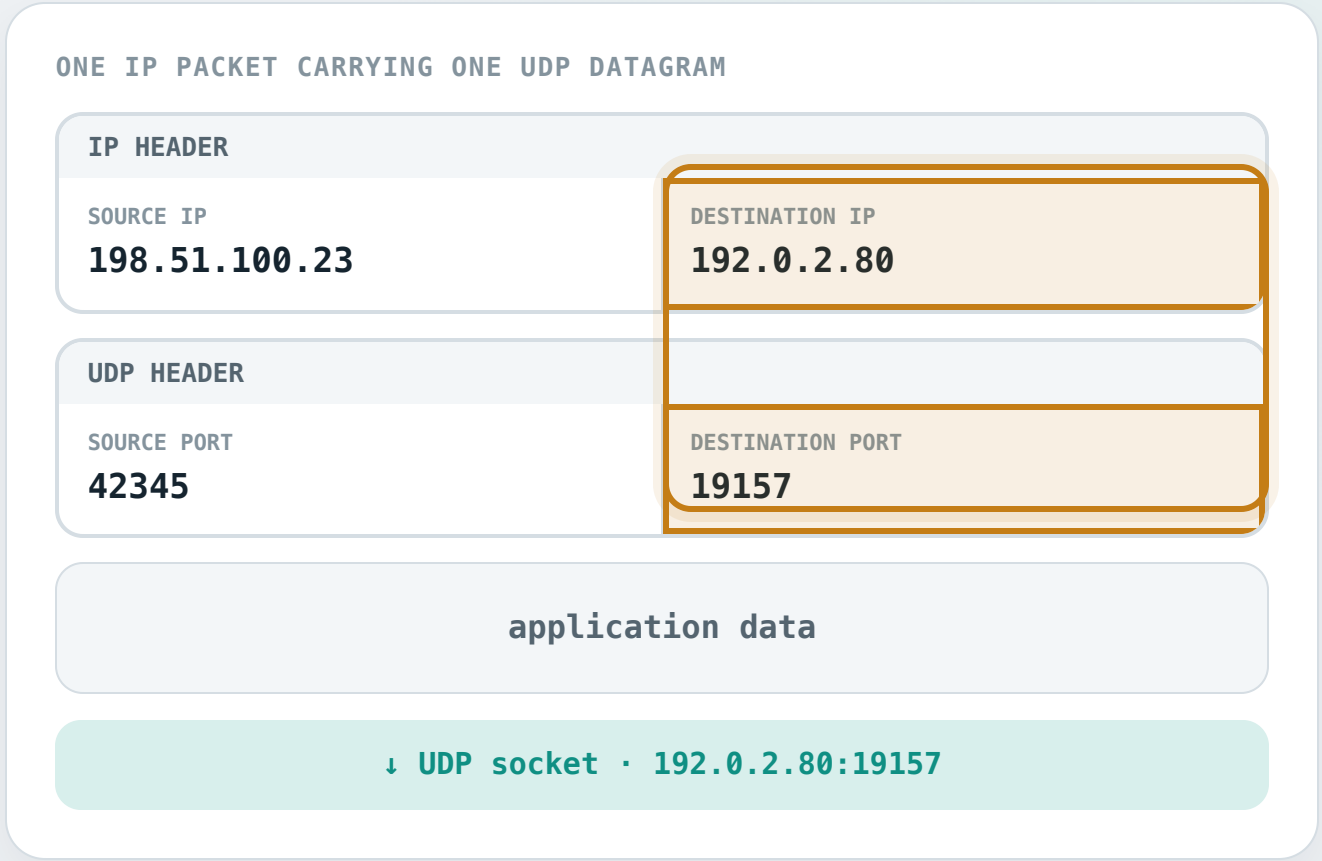


UDP demultiplexing

Suppose the network layer uses the destination IP address to deliver the following IP packet to some host.

How does UDP perform demultiplexing afterwards?

> **UDP looks at two fields.** UDP uses the **destination IP address** in the IP header and the **destination port** in the **UDP header** to identify the correct socket.



TCP demultiplexing

Suppose the network layer uses the destination IP address to deliver the following IP packet to some host.

How does TCP perform demultiplexing afterwards?

TCP looks at four fields. TCP uses the **source IP address** and **destination IP address** in the IP header, plus the **source port** and **destination port** in the **TCP header**, to identify the correct connected socket.

TCP CONNECTION IDENTITY

Source endpoint	Destination endpoint
SOURCE IP ADDRESS 203.0.113.10	DESTINATION IP ADDRESS 192.0.2.80
SOURCE PORT 46428	DESTINATION PORT 80

203.0.113.10:46428 ↔ 192.0.2.80:80

THE TWO TRANSPORT PROTOCOLS

Concept 2 of 4

UDP and TCP

One is connectionless and minimal; the other is connection-oriented and reliable.

User Datagram Protocol: an overview

UDP mechanism

UDP adds port-based multiplexing, demultiplexing, and error detection on top of IP.

Best-effort delivery

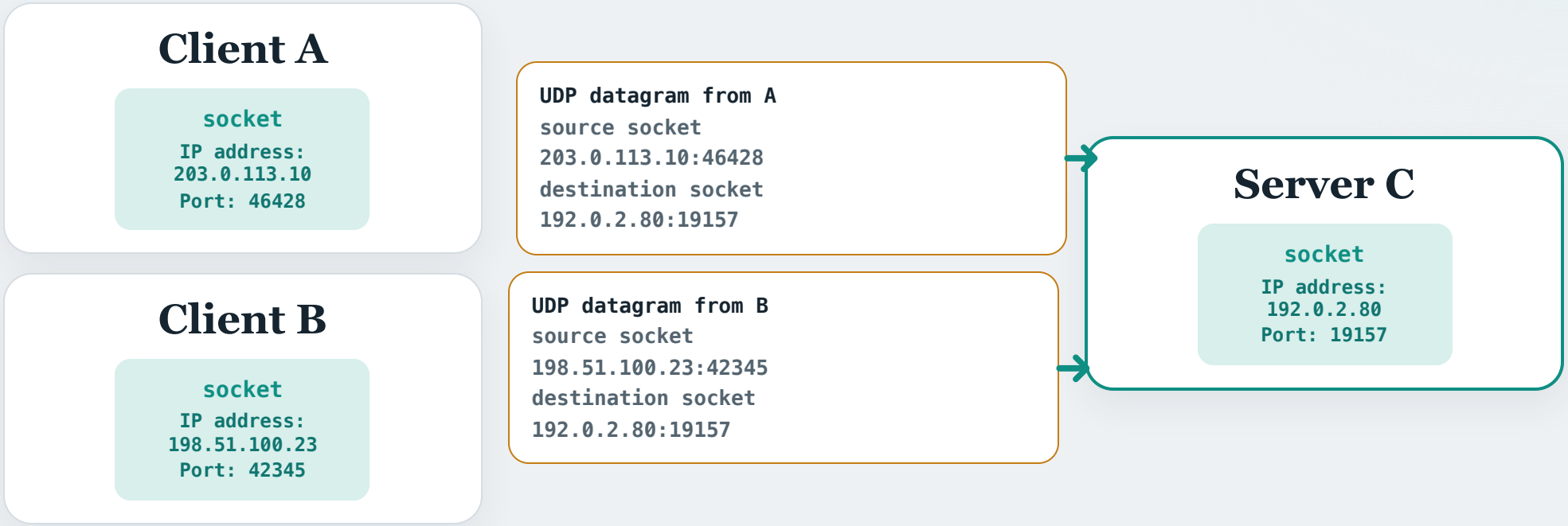
UDP does not guarantee delivery, ordering, or protection against duplicates. Applications are responsible for any recovery or ordering they need.

No connection setup

A host can send a UDP message without first establishing a transport-layer handshake.

UDP is connectionless

- Any sender can send a UDP datagram to another host without first establishing a connection.
- Different sources can send UDP datagrams to the same destination socket.
- In this case, the destination host can reply to each source socket by using the source IP address and port number carried in that UDP datagram.



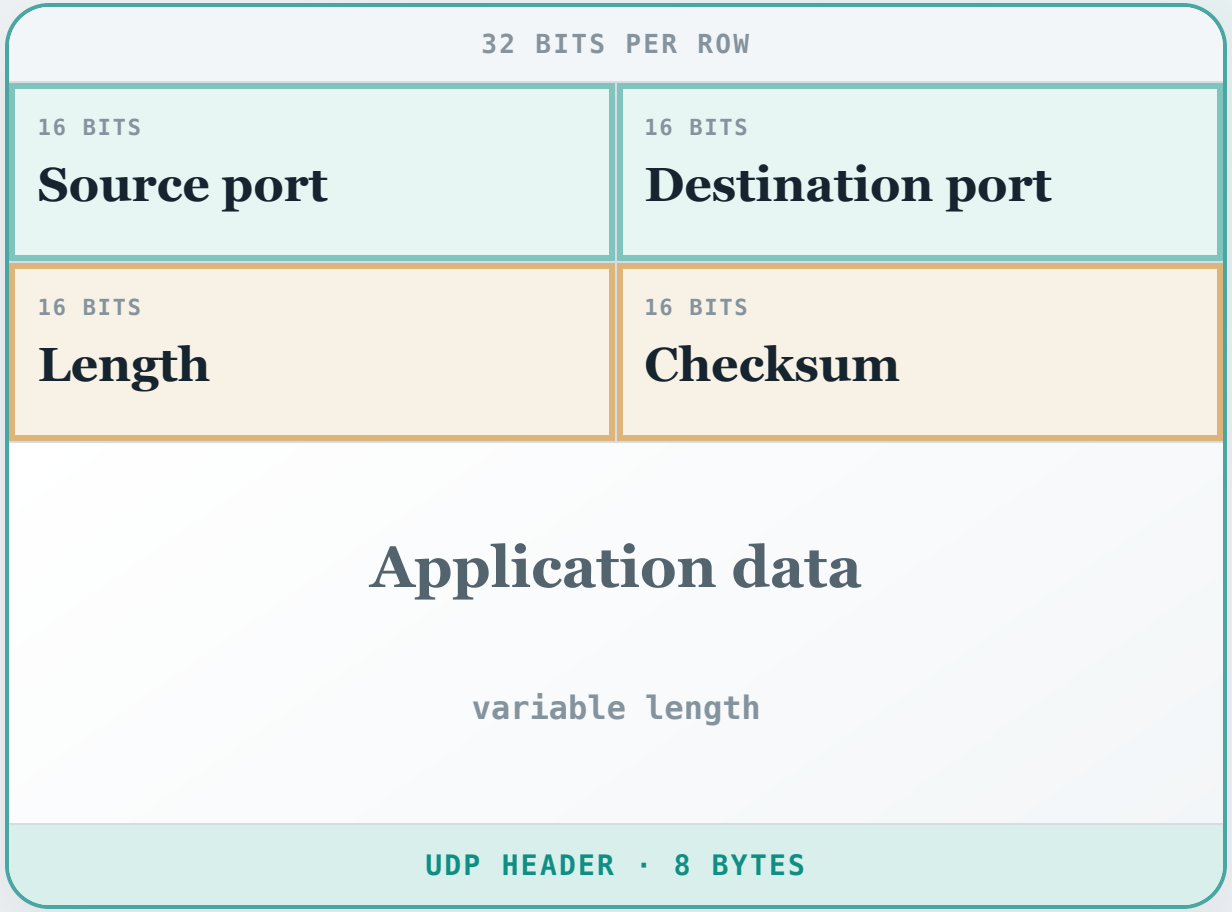
FOUR FIELDS · EIGHT-BYTE HEADER

A UDP segment has a compact header

Ports. The 16-bit source and destination ports identify the sending and receiving application endpoints.

Length. The total number of bytes in the UDP header and application data; the minimum is eight.

Checksum. Covers the UDP header and the entire payload data to detect corruption.



THE BENEFIT OF A MINIMAL TRANSPORT SERVICE

Why do some applications choose UDP?

1

No setup delay

An application can send immediately because UDP does not establish a transport connection first.

2

Minimal connection state

UDP does not maintain TCP-style connection buffers, sequence state, or windows.

3

Small header

Every UDP message adds only an eight-byte transport header.

4

Application control

The application decides when to send and can add only the recovery behavior it needs.

Common UDP examples include DNS lookups, real-time media, and online games.

TCP: an overview

Connection-oriented and point-to-point

TCP provides an end-to-end connection between exactly two endpoints, established with a handshake.

Byte stream and full duplex

TCP carries a continuous byte stream without application message boundaries, and both endpoints can send and receive simultaneously.

Reliable and ordered

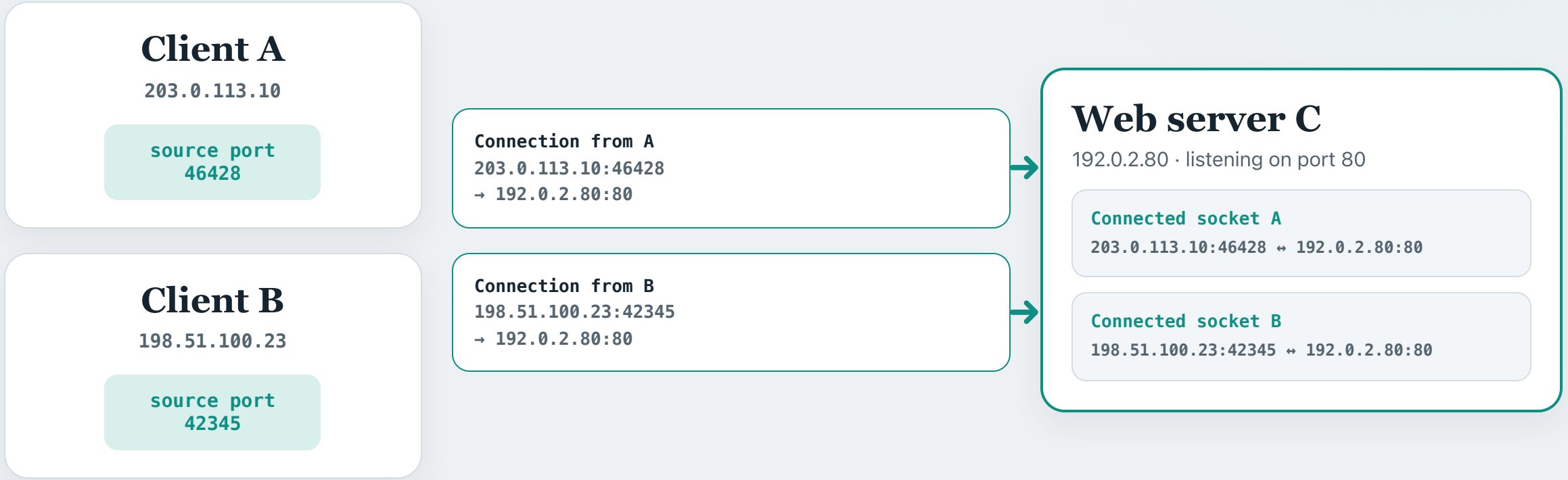
Unlike UDP's best-effort service, TCP delivers the byte stream reliably and in order.

Controlled pipelining

Like the pipelined RDT studied in Module 4, TCP keeps data in flight, subject to flow and congestion control.

TCP is connection-oriented

- TCP identifies each connection by four values: source IP address, source port, destination IP address, and destination port.
- Whereas UDP allows multiple clients to send to the same destination socket, TCP keeps each connection in a separate connected socket.



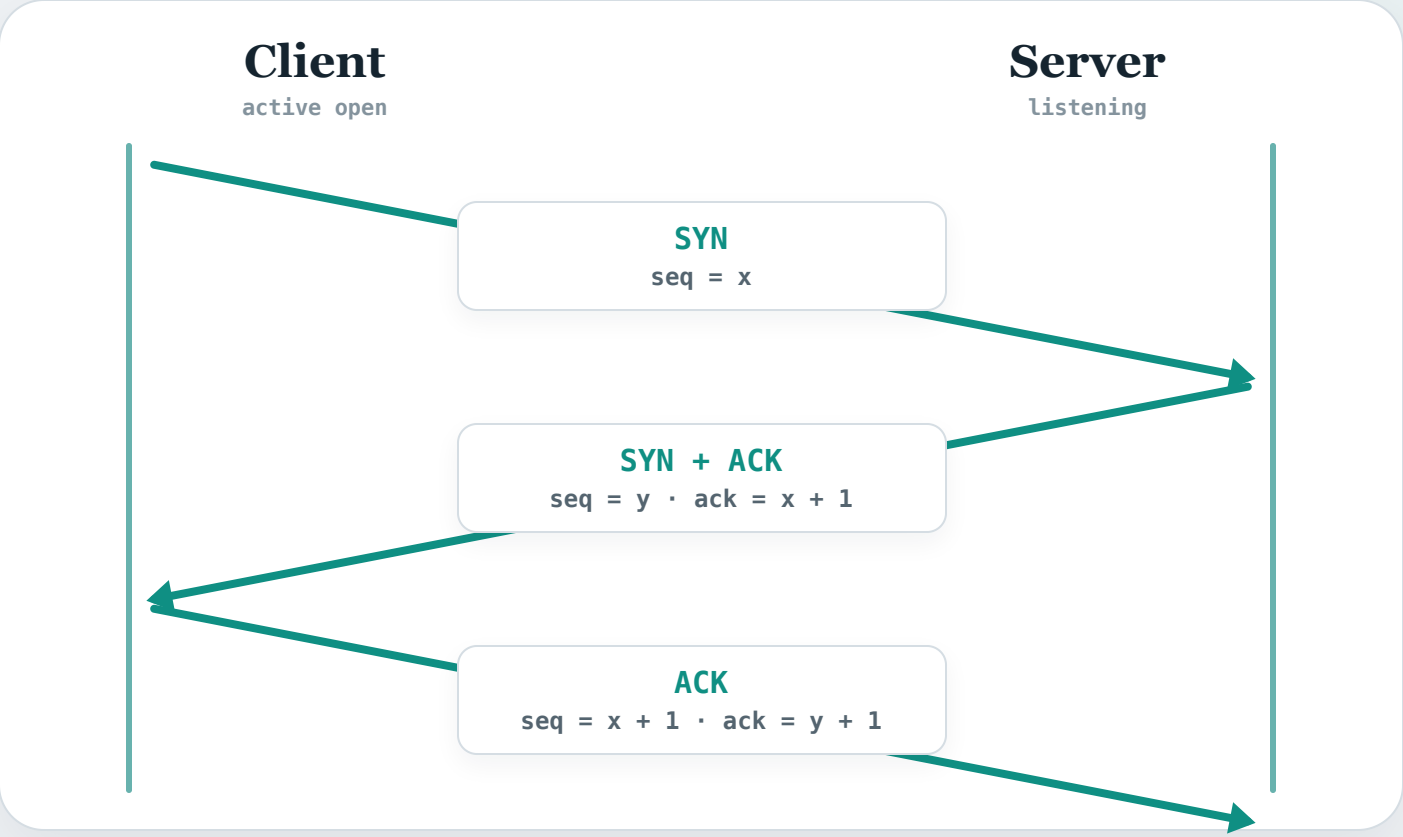
TCP establishes a connection with a three-way handshake

1 · SYN. The client chooses a random sequence number **x** and sends **SYN** with **seq = x**.

2 · SYN + ACK. The server chooses a random sequence number **y** and sends **SYN + ACK** with **seq = y** and **ack = x + 1**.

3 · ACK. The client sends **ACK** with **seq = x + 1** and **ack = y + 1**.

Recall Module 4 · RDT: TCP also uses sequence numbers and acknowledgments.



Now both endpoints have synchronized their sequence numbers and know the next number expected, so application data can flow.

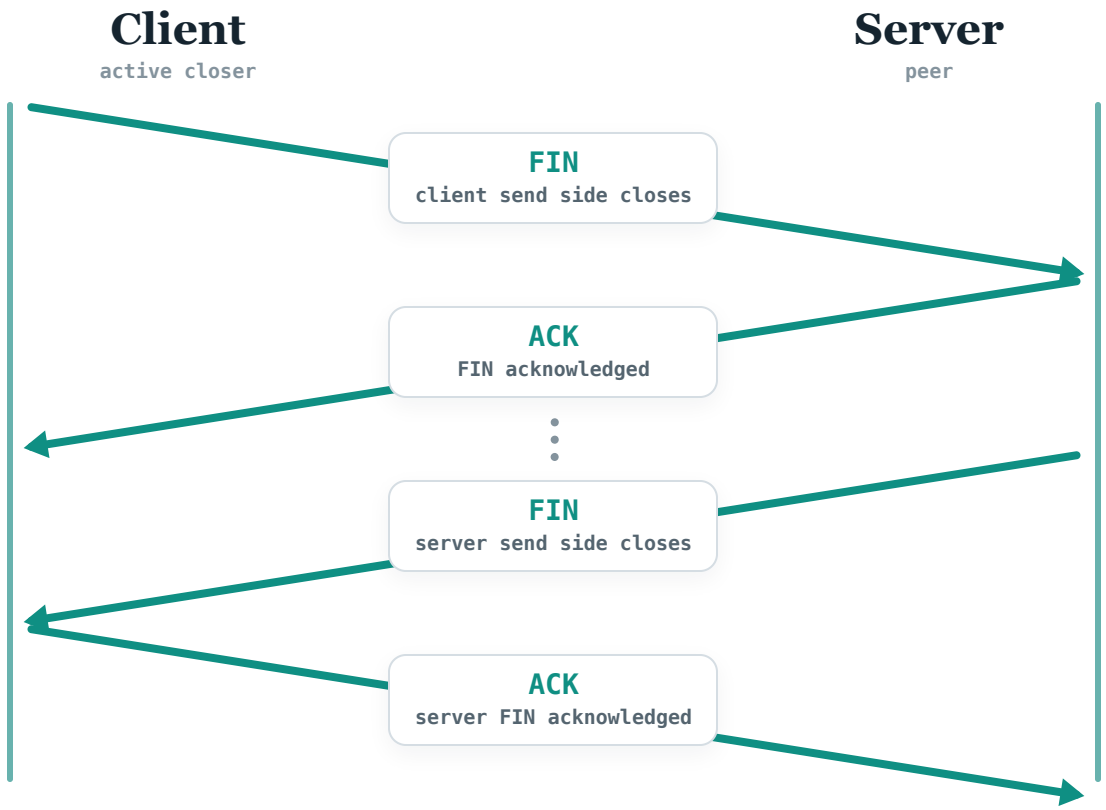
How TCP closes a connection

1 · FIN. The client sends **FIN** to say it has no more bytes to send.

2 · ACK. The server sends **ACK**; it may still send data before closing its direction.

3 · FIN. When the server is finished sending, it sends **FIN**.

4 · ACK. The client sends the final **ACK**, completing the close.



Both directions are closed, so the TCP connection is finished.

A COMPACT BREAKDOWN OF THE TCP HEADER

Key TCP header fields

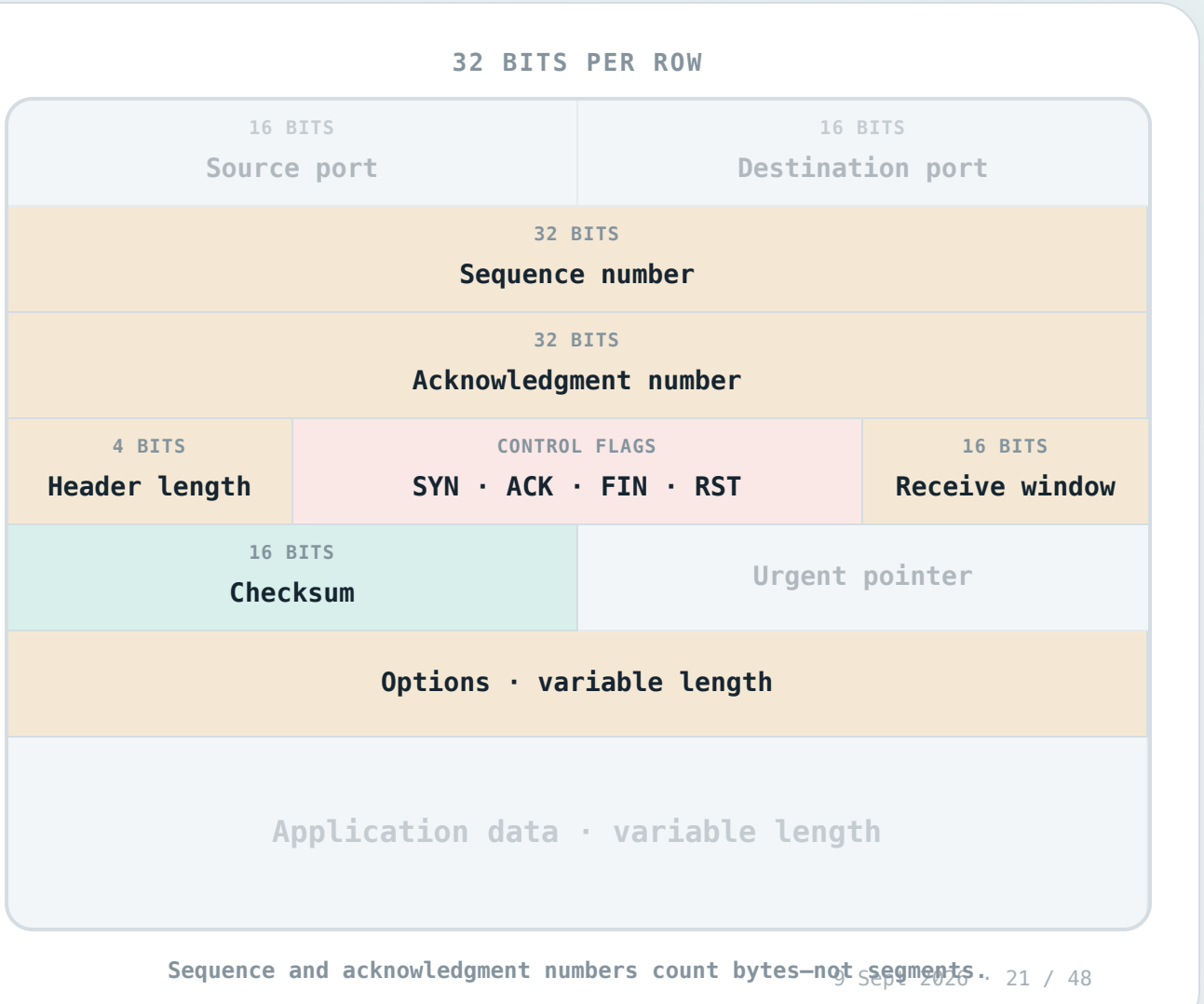
Sequence and acknowledgment numbers. They identify the first byte sent and the next byte expected.

Control flags. Each is a one-bit field; a segment sets the relevant bit to 1 to signal an action such as SYN, ACK, FIN, or RST.

Receive window. It tells the peer how many more data bytes the receiver can accept.

Checksum. It detects corruption in the TCP header and payload.

Header length and options. Header length shows where data begins; options carry extra capabilities such as MSS.



Sequence and acknowledgment numbers count bytes—not segments.

The RDT used by TCP — an overview

TCP uses a reliable data transfer protocol on top of IP to provide reliable, in-order byte delivery.

Its RDT is inspired by both **Go-Back-N** and **Selective Repeat** — the pipelined protocols we studied in Week 10.

Pipelining and cumulative acknowledgments

The TCP sender uses a **window** of segments in flight, and the TCP receiver uses **cumulative ACKs**.

Unlike Go-Back-N — where the ACK names the last in-order segment received — TCP's ACK names the **next byte expected**.

Loss recovery

Retransmission happens when the timer times out or when duplicate ACKs arrive (**fast retransmit**).

TCP sequence numbers count bytes

Byte-level addressing

In the RDT protocols from Week 10, each sequence number advanced by one **packet** — between consecutive sender messages, the sequence number advanced by one.

In TCP, sequence numbers address individual **bytes** — so between consecutive sender segments, the sequence number advances by the **number of bytes** in the segment.

Maximum Segment Size (MSS)

The largest payload a single TCP segment can carry — negotiated during the handshake. Typical value: **1460 bytes** on Ethernet (1500 MTU – 20 IP – 20 TCP).

SYN and FIN consume a sequence number

SYN and FIN each consume exactly one sequence number, even though they carry no application data — this ensures they can be reliably acknowledged.

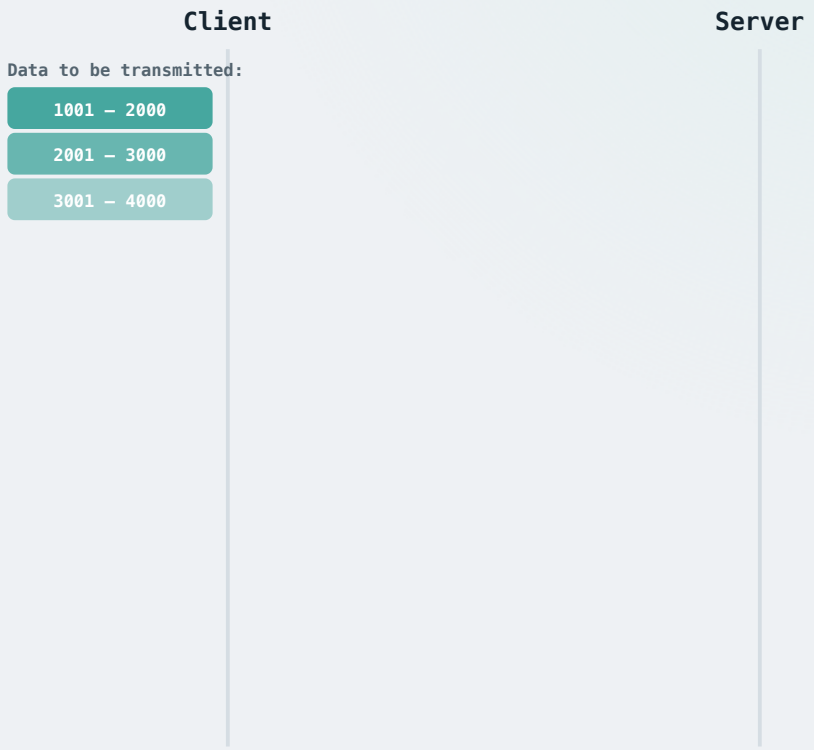
ISN = 1000 · MSS = 1000 BYTES · 3000 BYTES TO SEND

Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

The setup

With MSS = 1000 bytes, the 3000 bytes of data fill **three segments**.



ISN = 1000 · MSS = 1000 BYTES · 3000 BYTES TO SEND

Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

Three-way handshake

First, the client and server perform a three-way handshake to open the TCP connection.

Suppose ISN = 1000. Since SYN consumes one sequence number, the first data byte will have seq# **1001**.

The server's SYNACK carries **ACK = 1001**, which signals that it expects byte 1001 next.

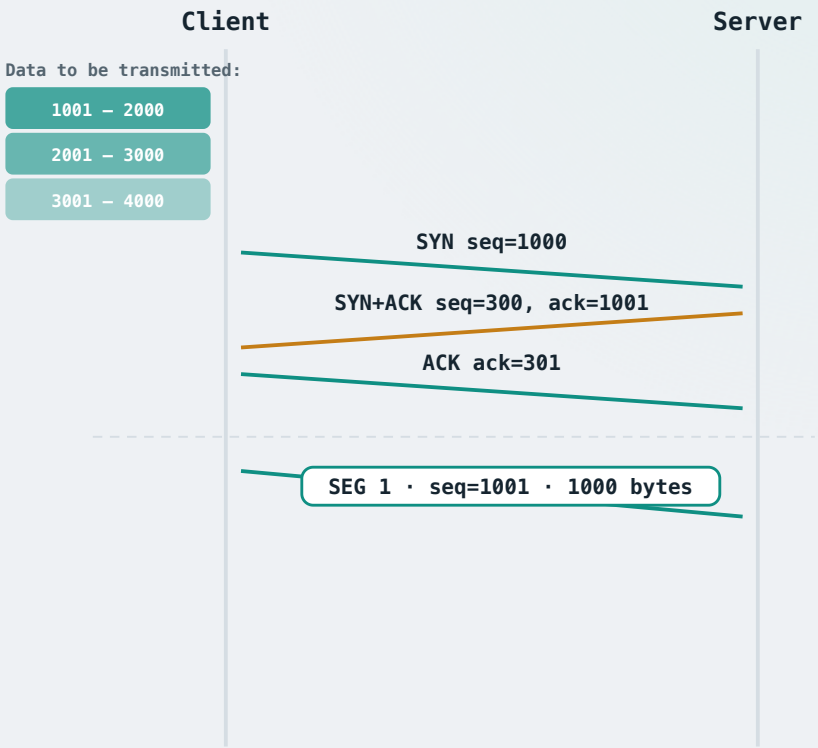
ISN = 1000 · MSS = 1000 BYTES · 3000 BYTES TO SEND

Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

Transmit segment 1

The client sends the first 1000 bytes (seq = 1001).



ISN = 1000 · MSS = 1000 BYTES · 3000 BYTES TO SEND

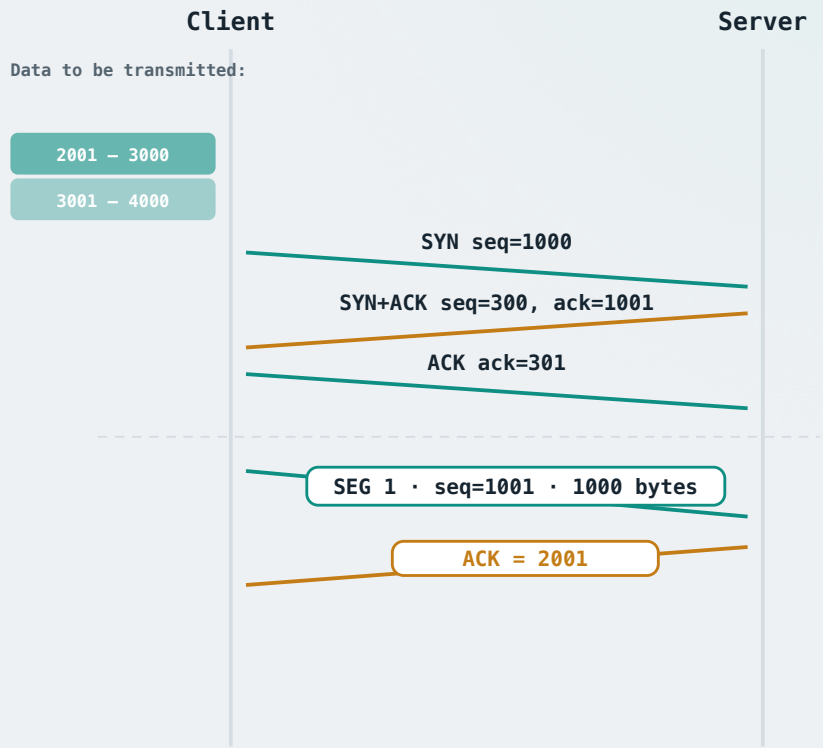
Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

Transmit segment 1

The client sends the first 1000 bytes (seq = 1001).

The server cumulatively acknowledges with **ACK = 2001** — it has every byte up to 2000 and expects byte 2001 next.



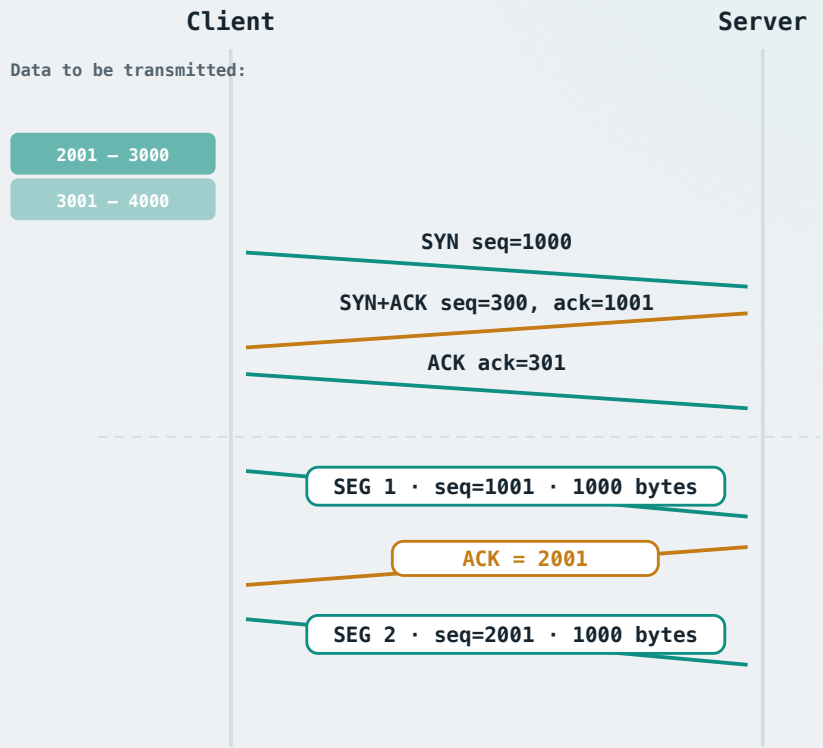
ISN = 1000 · MSS = 1000 BYTES · 3000 BYTES TO SEND

Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

Transmit segment 2

The client sends the next 1000 bytes (seq = 2001).



ISN = 1000 · MSS = 1000 BYTES · 3000 BYTES TO SEND

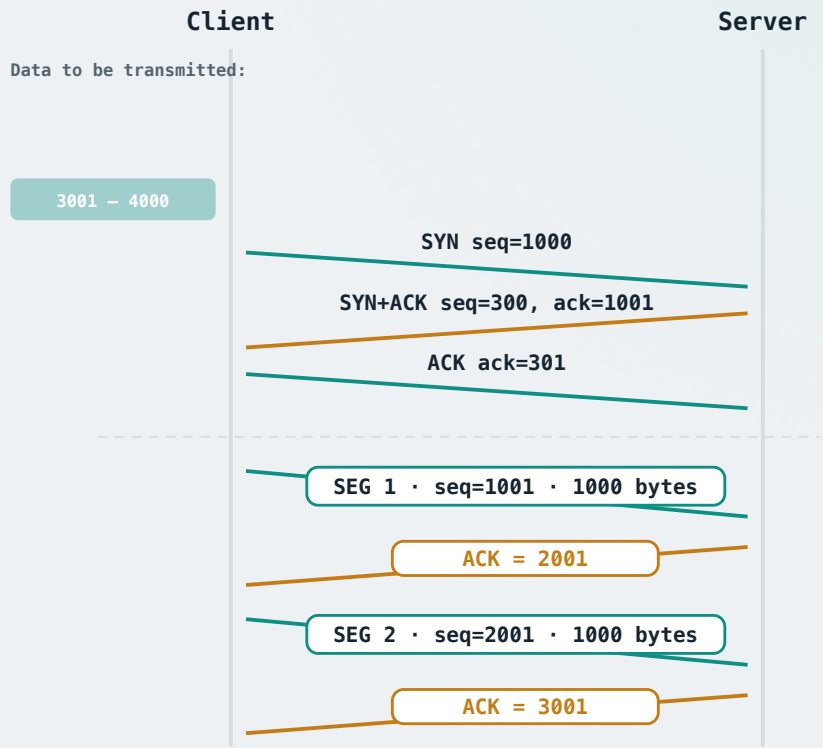
Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

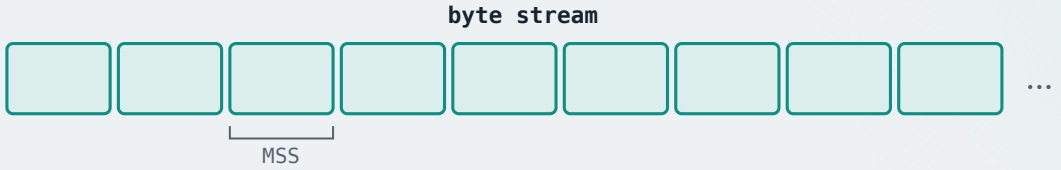
Transmit segment 2

The client sends the next 1000 bytes (seq = 2001).

The server cumulatively acknowledges with **ACK = 3001** — confirming every byte up to 3000.



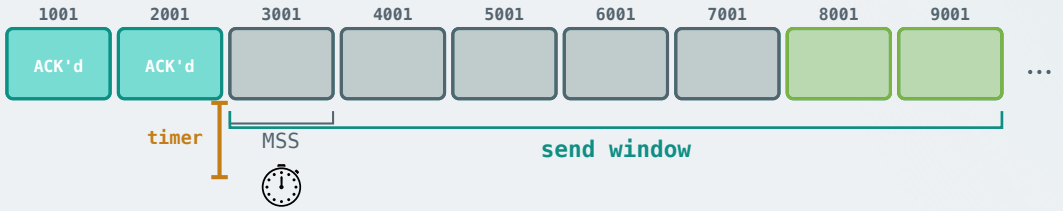
The TCP sender rules: overview



Data from application

Divide the byte stream into segments, typically **1 MSS** bytes each.

The TCP sender rules: overview



Data from application

Divide the byte stream into segments, typically **1 MSS** bytes each.

Assign the next sequence number to each segment and transmit it if within the **send window**.

e.g. window base is seq 3001; the first 5 segments (3001–7001) are transmitted.

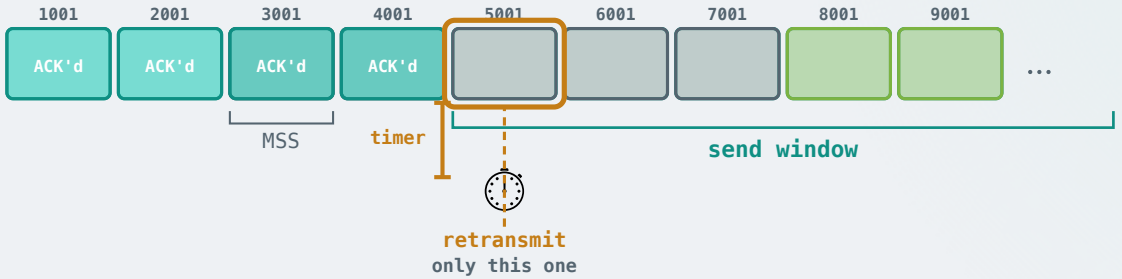
Start **one timer** for the window if none is running.

ACK received

A cumulative ACK slides the window forward.

Restart the timer if unacknowledged data remains; stop it if everything is confirmed.

The TCP sender rules: overview



Data from application

Divide the byte stream into segments, typically **1 MSS** bytes each.

Assign the next sequence number to each segment and transmit it if within the **send window**.

e.g. window base is seq 3001; the first 5 segments (3001–7001) are transmitted.

Start **one timer** for the window if none is running.

ACK received

A cumulative ACK slides the window forward.

Restart the timer if unacknowledged data remains; stop it if everything is confirmed.

e.g. ACK 5001 received → segs 3001 and 4001 now ACKed; base advances, window slides, timer restarts on 5001.

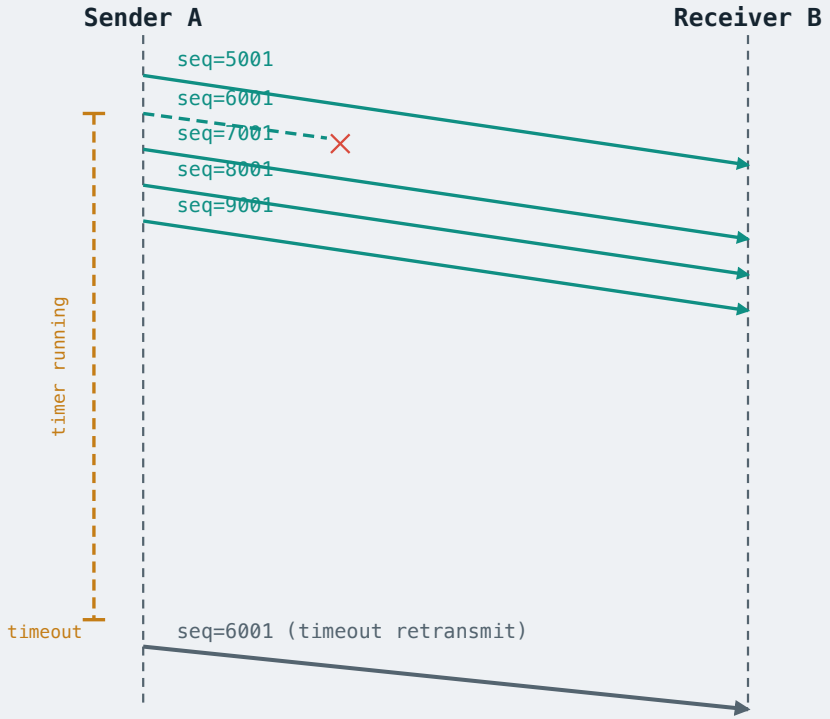
Timeout

Retransmit only the **oldest unacknowledged segment**, then restart the timer.

Different from Go-Back-N: GBN retransmits *every* in-flight packet in the window. TCP retransmits only *one*.

e.g. in this example, GBN would retransmit segs 5001, 6001, and 7001; TCP retransmits only seg 5001.

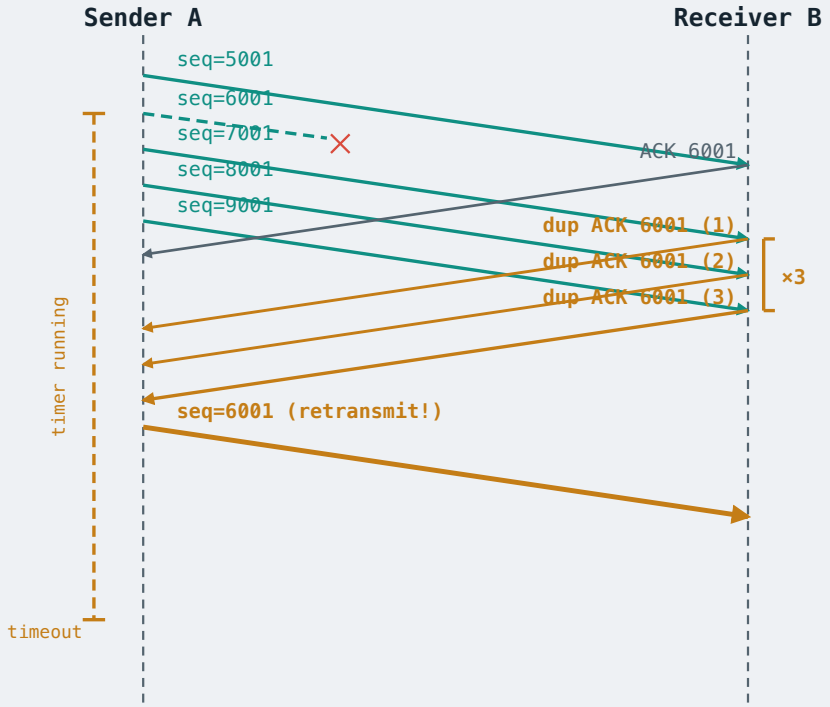
TCP fast retransmit



The problem

Timeout period is relatively long. Suppose seg 6001 is lost — in Go-Back-N, the sender must wait until timeout before it can retransmit. This can take a long time.

TCP fast retransmit



The problem

Timeout period is relatively long. Suppose seg 6001 is lost — in Go-Back-N, the sender must wait until timeout before it can retransmit. This can take a long time.

Duplicate ACKs — an early signal

The receiver ACKs the last in-order byte. When 7001, 8001, and 9001 arrive but 6001 is missing, each generates another dup ACK 6001.

A stuck ACK (not advancing) means the receiver is still waiting for the same segment — already an *early indication of loss*, before the timer fires.

Fast retransmit

After **3 duplicate ACKs**, retransmit the missing segment immediately — without waiting for the timeout.

TCP receiver rules: overview

Like Go-Back-N: cumulative ACK

The receiver sends a **cumulative ACK** for the highest in-order byte received. If a segment arrives out of order, the receiver sends a **duplicate ACK** — ACKing the next expected sequence number, signalling the gap to the sender.

Like Selective Repeat: receiver buffer

Unlike Go-Back-N, the receiver **buffers out-of-order segments** within the receive window rather than discarding them. Once the missing segment arrives and the gap is filled, buffered data is delivered to the application in order.

Further improvements

There are further recommendations for ACK generation (e.g. to avoid sending too many ACKs) in **RFC 1122** and **RFC 2581**.

TCP can also negotiate **selective ACK options** [**RFC 2018**] — the receiver reports exactly which byte ranges it holds, letting the sender retransmit only the missing pieces, much like Selective Repeat.

TCP timeout: the challenge

The timer should track RTT

The sender's retransmit timer is set to roughly the **round-trip time** —

just long enough for a packet to receive its ACK. Too short → spurious retransmissions waste bandwidth. Too long → slow recovery when a segment really is lost.

The problem: RTT is not constant

Network load, queueing, and path changes cause RTT to fluctuate — sometimes dramatically.

A fixed timeout value cannot adapt. TCP must **estimate RTT dynamically** and update the timer continuously.

TCP RTT estimation

SampleRTT

EstimatedRTT is derived by taking many individual RTT samples.

Each sample is measured from when a segment is transmitted until its ACK is received.

Retransmitted segments are **excluded**: the ACK could be for the original send or the retransmit — the sender cannot tell, so the measurement would be unreliable.

EstimatedRTT — exponential weighted moving average

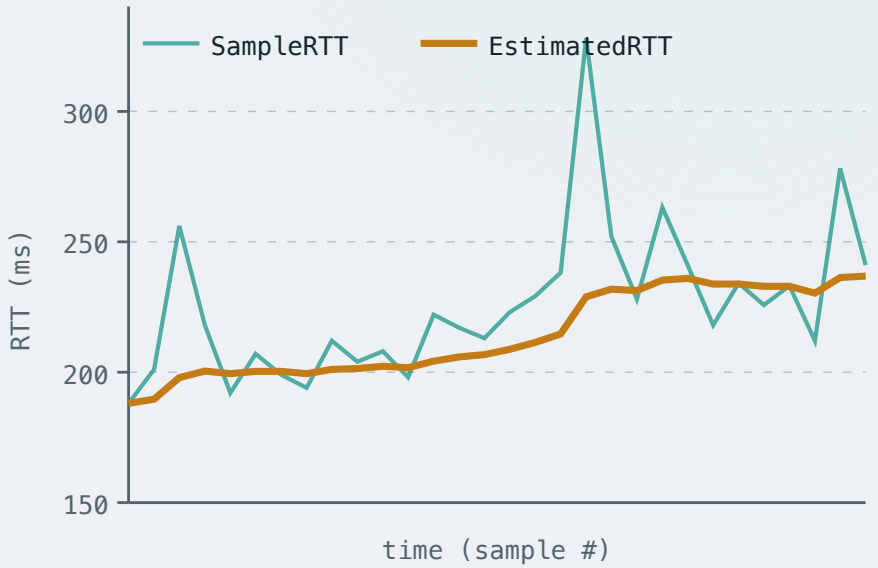
SampleRTT varies noticeably from measurement to measurement — so TCP uses an averaging technique to compute a smooth, adaptive estimate.

$$\text{EstimatedRTT} = (1-\alpha)\cdot\text{EstimatedRTT} + \alpha\cdot\text{SampleRTT}$$

Typical value: **$\alpha = 0.125$**. Unrolling this one step at a time:

$$\text{EstimatedRTT} = \alpha\cdot\text{Sample}_n + \alpha(1-\alpha)\cdot\text{Sample}_{n-1} + \alpha(1-\alpha)^2\cdot\text{Sample}_{n-2} + \ldots$$

Each step further back adds another factor of (1–α) = 0.875 — so older samples count exponentially less.



TCP timeout interval

EstimatedRTT alone is not enough

It might be tempting to just set the timeout interval to EstimatedRTT.

However, this provides no safety margin for RTT variability — when RTT spikes, the timer fires too early.

DevRTT — tracking variability

Therefore we also maintain **DevRTT** — a tracker for how much SampleRTT deviates from EstimatedRTT.

$$DevRTT = (1-\beta)\cdot DevRTT + \beta\cdot|SampleRTT - EstimatedRTT|$$

Typical value: $\beta = 0.25$.

TimeoutInterval

$$TimeoutInterval = EstimatedRTT + 4\cdot DevRTT$$

Empirically, this prevents the timer from firing too early — for example, when sudden network congestion causes a spike in RTT.

MATCHING THE SENDER TO THE RECEIVER

Concept 3 of 4

TCP Flow Control

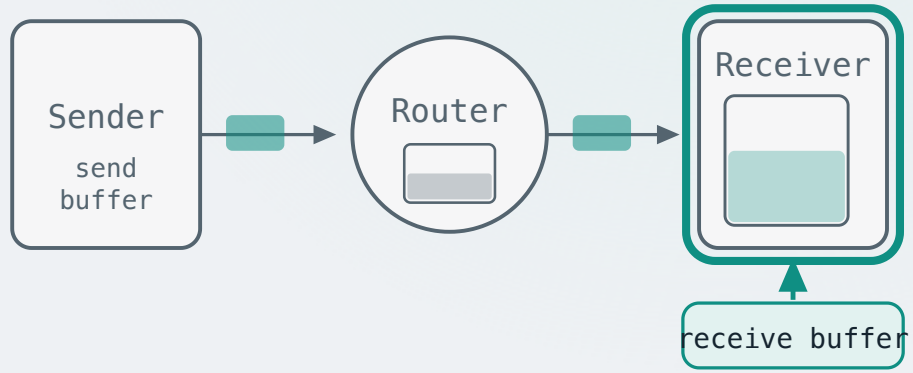
TCP uses receiver feedback so that a fast sender cannot swamp a slow receiver.

Flow control vs congestion control

Flow control

The receiver maintains a buffer to hold incoming segments before the application reads them — if the sender transmits too fast, this buffer overflows and segments are lost.

Flow control prevents this by matching the sender's transmission rate to the speed at which the receiver's application is reading.



Flow control vs congestion control

Flow control

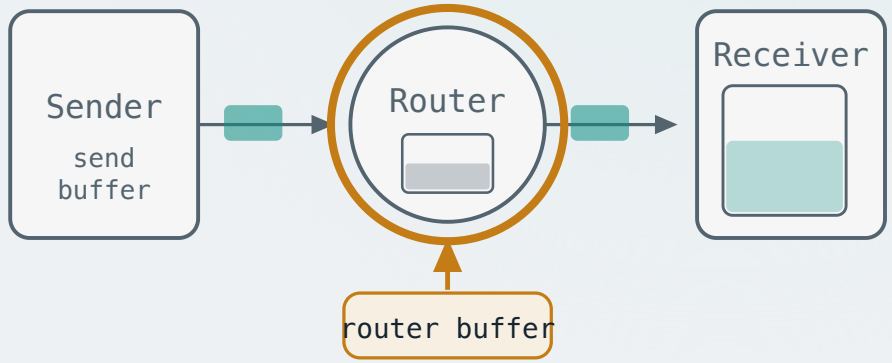
The receiver maintains a buffer to hold incoming segments before the application reads them — if the sender transmits too fast, this buffer overflows and segments are lost.

Flow control prevents this by matching the sender's transmission rate to the speed at which the receiver's application is reading.

Congestion control

Segments travel through shared routers, each with finite buffers — if too many senders transmit simultaneously, router buffers fill up and segments are dropped.

Congestion control limits each sender's rate to prevent this network overload.



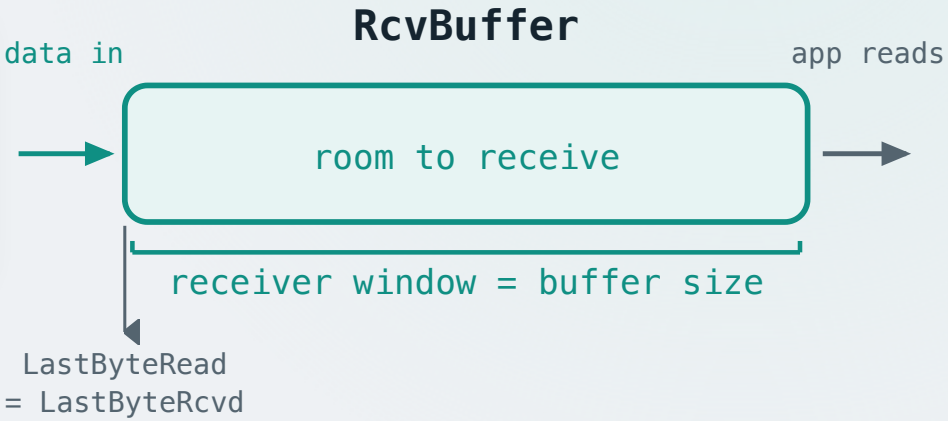
SPEED-MATCHING THE SENDER TO THE RECEIVER

TCP flow control: the receive window

The receive buffer and floating window

The receiver allocates a fixed-size receive buffer (**RcvBuffer**) for the connection. Incoming data sits there until the application reads it.

In Selective Repeat, we assume the application consumes data as fast as it arrives — the buffer is always fully available, so **receiver window = buffer size**, always.



TCP flow control: the receive window

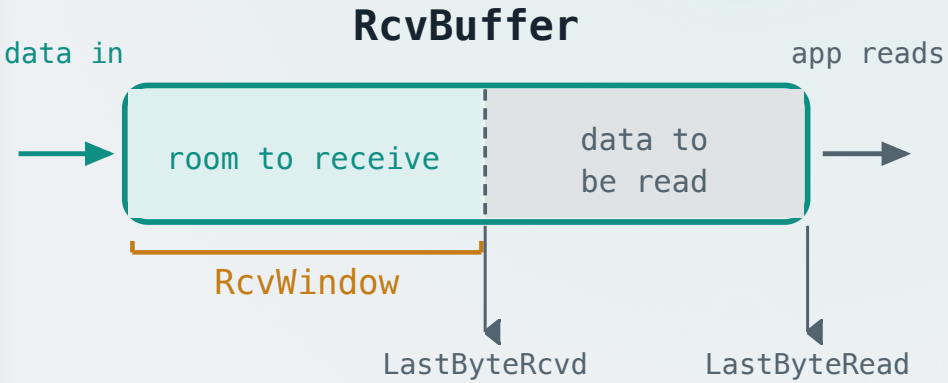
The receive buffer and floating window

The receiver allocates a fixed-size receive buffer (**RcvBuffer**) for the connection. Incoming data sits there until the application reads it.

In Selective Repeat, we assume the application consumes data as fast as it arrives — the buffer is always fully available, so **receiver window = buffer size**, always.

In TCP, the application reads at its own pace — so the buffer may contain unread data that reduces available space. We can identify that unread portion as [LastByteRcvd – LastByteRead]; thus the receiver window **floats**:

```
RcvWindow = RcvBuffer – [LastByteRcvd – LastByteRead]
```



FROM RECEIVER TO SENDER

Advertising and enforcing the receive window

Advertising the window

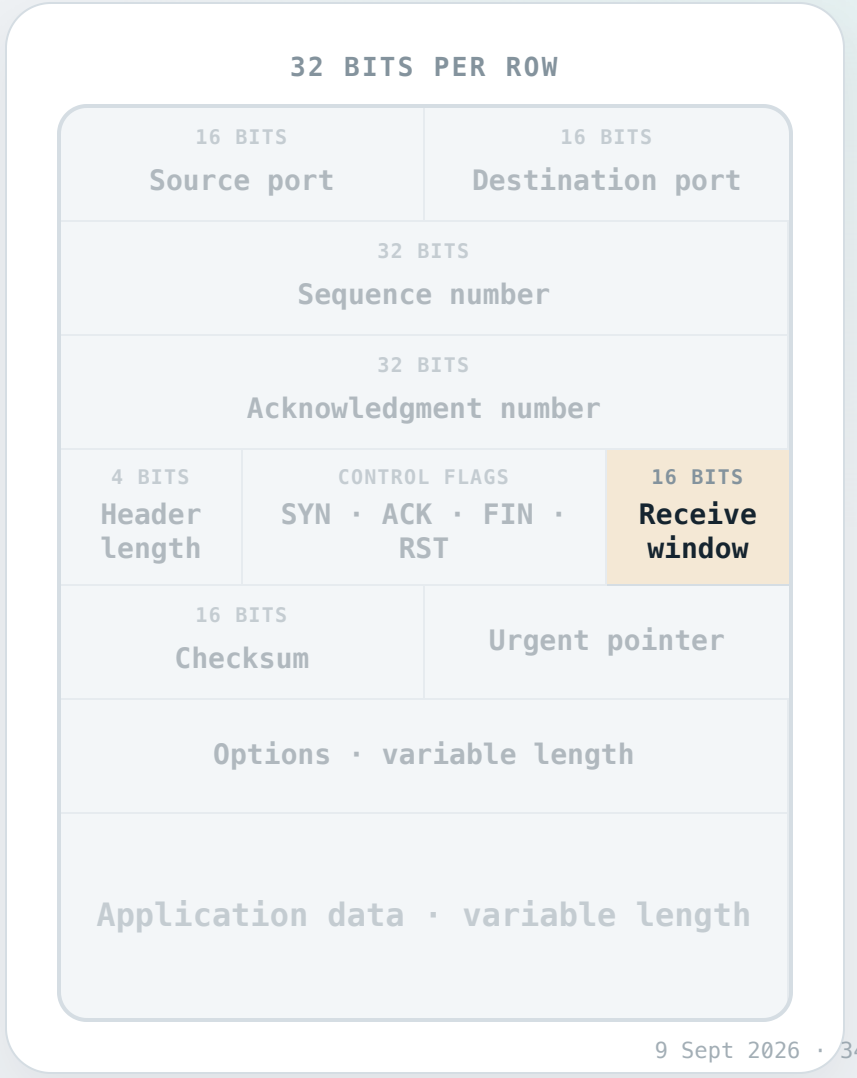
So how does the receiver advertise its current window to the sender?

The receiver piggybacks RcvWindow in every segment it sends, using the **Receive Window** field in the TCP header.

The sender's rule

`LastByteSent − LastByteAcked ≤ RcvWindow`

In other words, the sender ensures the amount of data in flight never exceeds the advertised RcvWindow.



MATCHING THE SENDER TO THE NETWORK

Concept 4 of 4

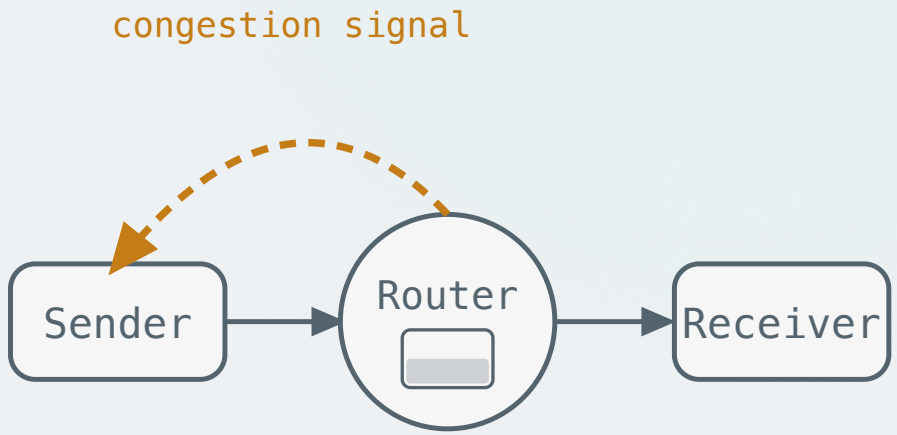
Congestion Control

Too many senders overload router buffers. TCP limits each sender's rate to prevent this.

Approaches to congestion control

Network-assisted

Routers explicitly notify senders to slow down — for example, by setting a congestion bit in a packet header, or stating the supported rate. Requires router cooperation throughout the path.



Approaches to congestion control

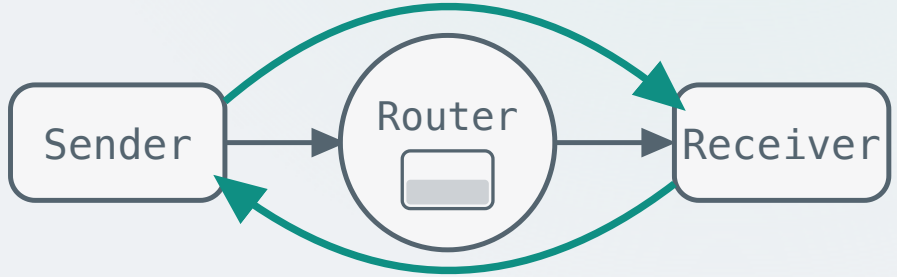
Network-assisted

Routers explicitly notify senders to slow down — for example, by setting a congestion bit in a packet header, or stating the supported rate. Requires router cooperation throughout the path.

End-to-end

No explicit feedback from the network. The sender infers congestion from observable signals — primarily **packet loss** and increased **delay**.

TCP uses the **end-to-end** approach — congestion is inferred from **loss events** in the network.



TCP congestion detection and action: an overview

Detection

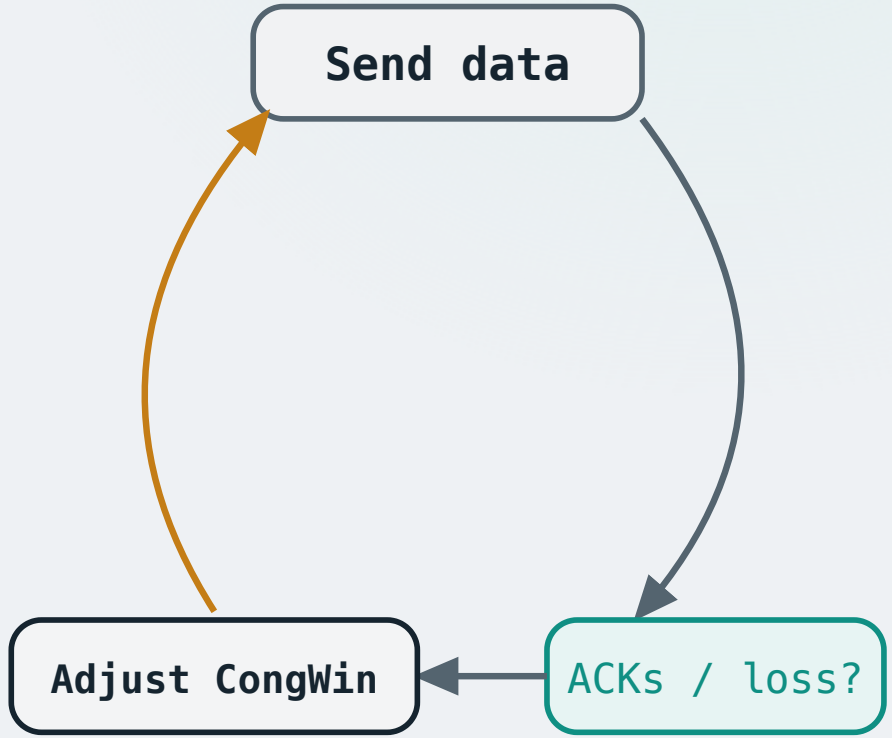
3 duplicate ACKs — one segment lost, but others still arriving (mild congestion).

Timeout — **nothing** gets through; the network appears unresponsive (serious congestion).

Action

The sender maintains a **congestion window (CongWin)** — a cap on how much unacknowledged data it may have in flight at any time.

On detecting congestion, **shrink** CongWin.



How TCP matches its sending rate

The in-flight constraint

```
LastByteSent − LastByteAcked ≤ CongWin
```

The sender always keeps its unacknowledged data in flight below this limit.

Per-RTT implication

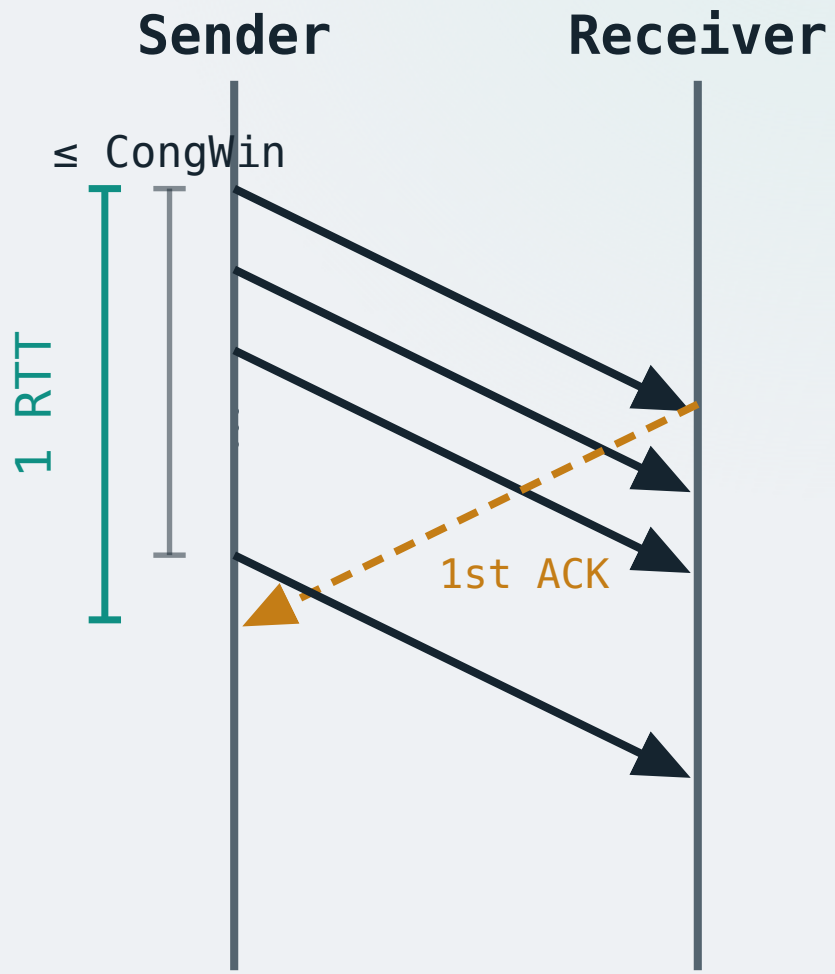
The sender transmits segments back-to-back — all remain in flight until the first ACK returns, which takes one RTT.

So within one RTT, everything sent is simultaneously in flight. Since the constraint says at most CongWin bytes can be in flight, **the sender can send at most CongWin bytes per RTT.**

The corresponding sending rate

Since TCP carries a continuous byte stream, the corresponding sending rate is:

```
sending rate ≈ CongWin / RTT  bytes/sec
```



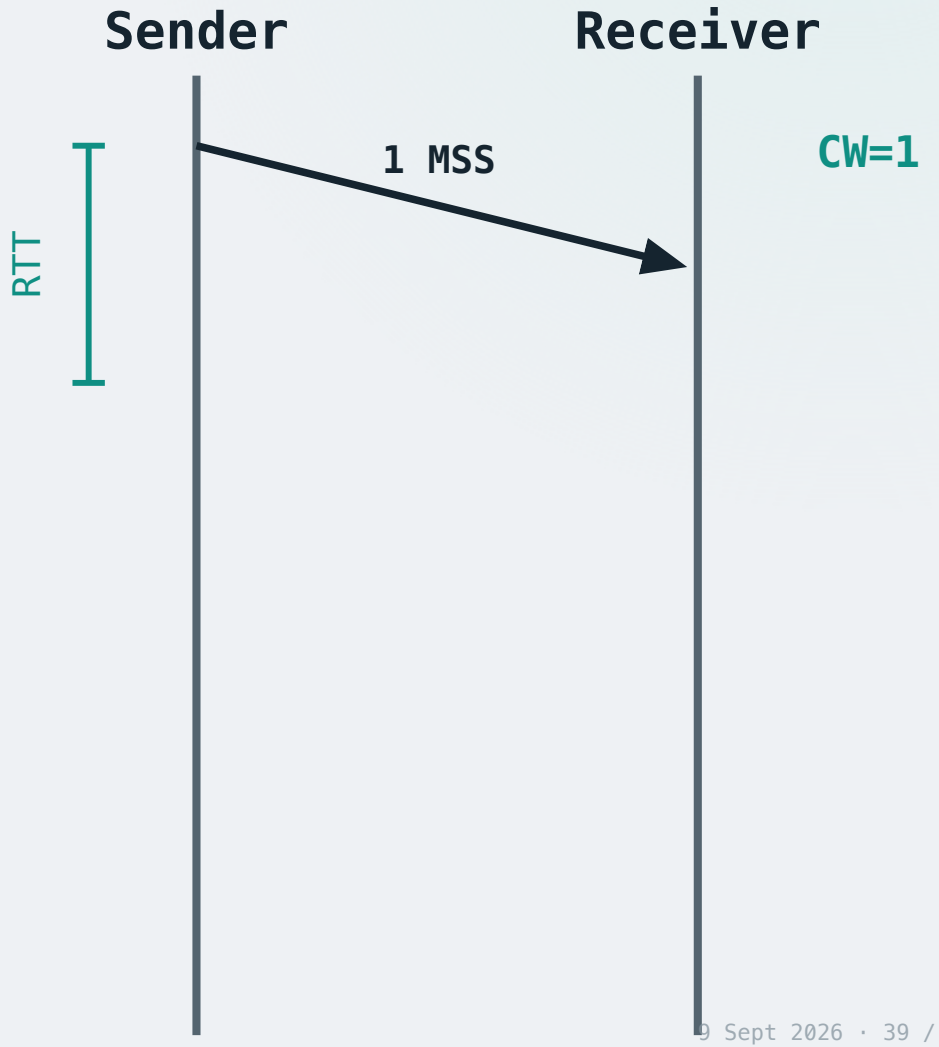
TCP · slow start

Starting conservatively

When a connection begins, the sender knows nothing about the path — start conservatively: **CongWin = 1 MSS**.

Since CongWin = 1 MSS, the in-flight constraint means the sender may only send **1 MSS per RTT**.

Each segment carries exactly 1 MSS of data. Send one segment, wait for its ACK.



TCP · slow start

Growing the window

Each ACK confirms the network handled the segment — likely no congestion yet.

SLOW-START RULE

For every ACK received: CongWin += 1 MSS

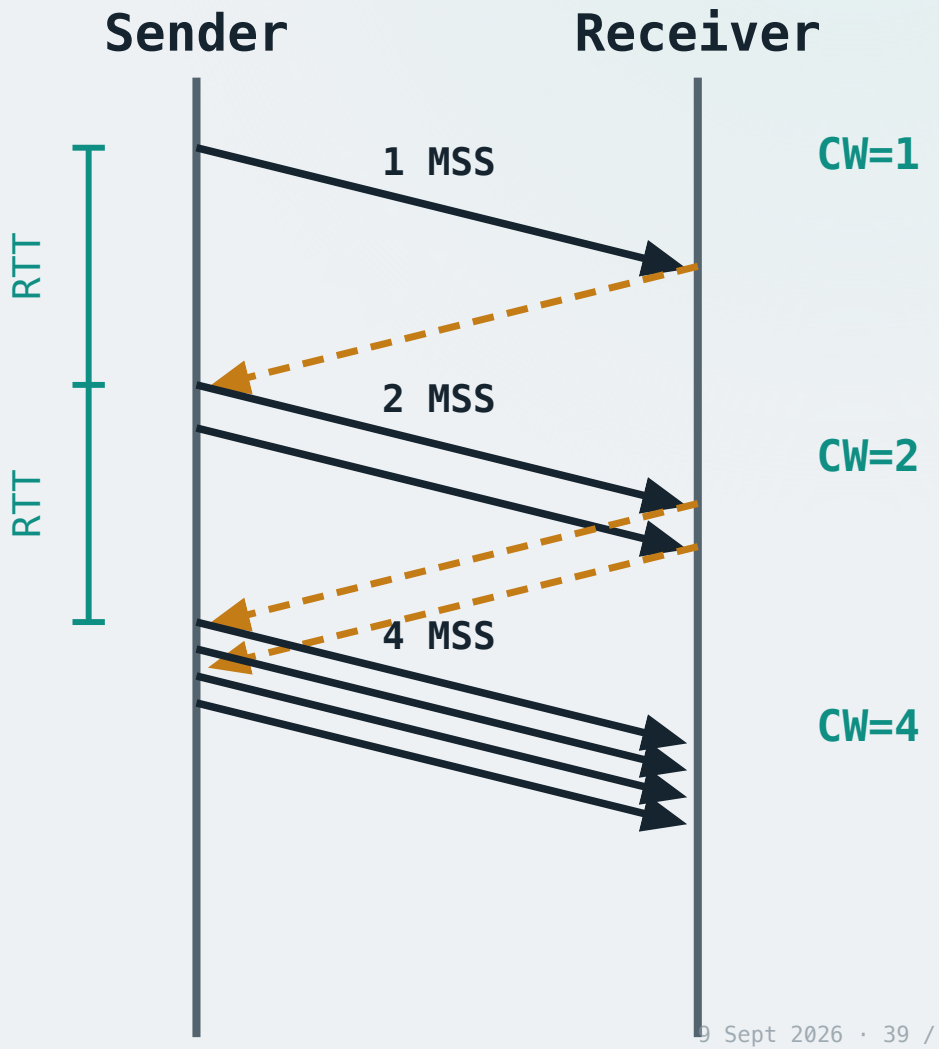
CongWin increases to 2 — in the next RTT the sender can send **2 MSS**.

Two more ACKs received — each adds 1 MSS. CongWin increases to 4, and in the next RTT we can send **4 MSS**.

The pattern: the growth rate depends on the current window size.

SLOW-START EMPIRICAL RULE

CongWin doubles every RTT — 1, 2, 4, 8, 16… — exponential growth

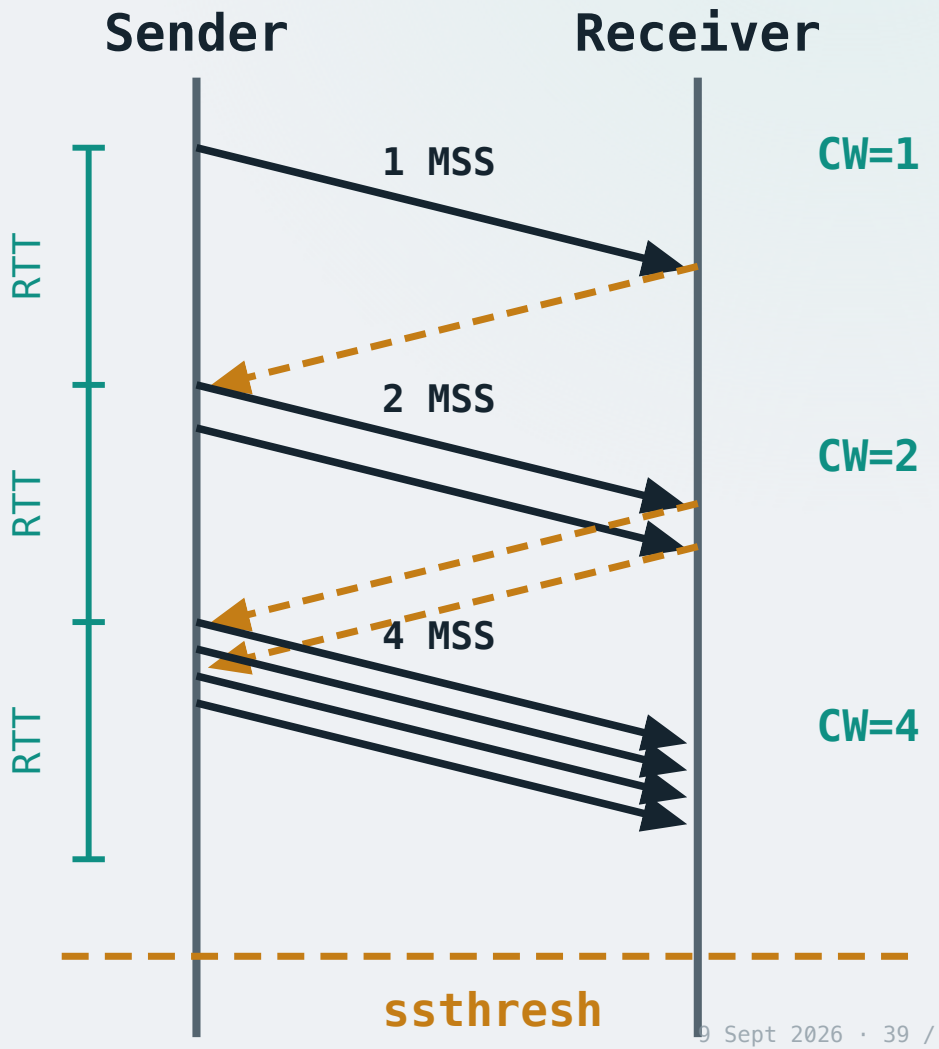


TCP · slow start

The threshold

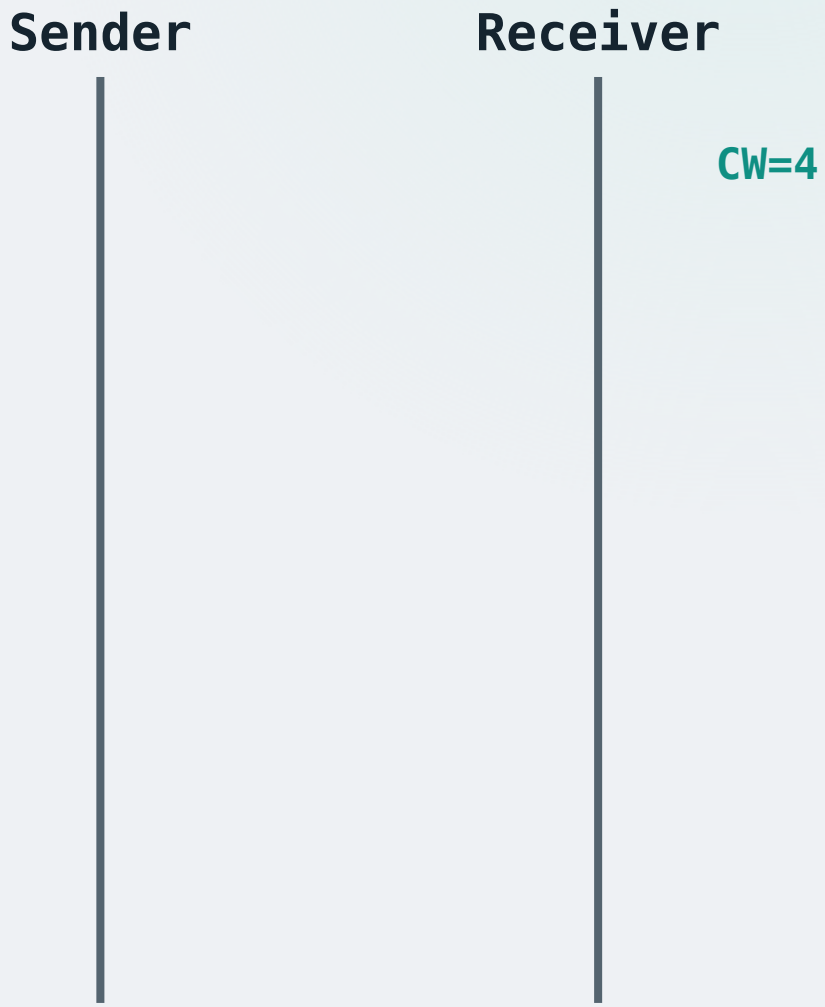
Exponential growth can't last forever. A threshold called **ssthresh** (slow-start threshold) marks where slow start ends.

When CongWin ≥ ssthresh → switch to **congestion avoidance** (next frame).



TCP · congestion avoidance

Suppose **ssthresh = 4 MSS** — the point where congestion previously occurred.



TCP · congestion avoidance

Additive increase

Once CongWin ≥ ssthresh, growth switches from exponential to linear.

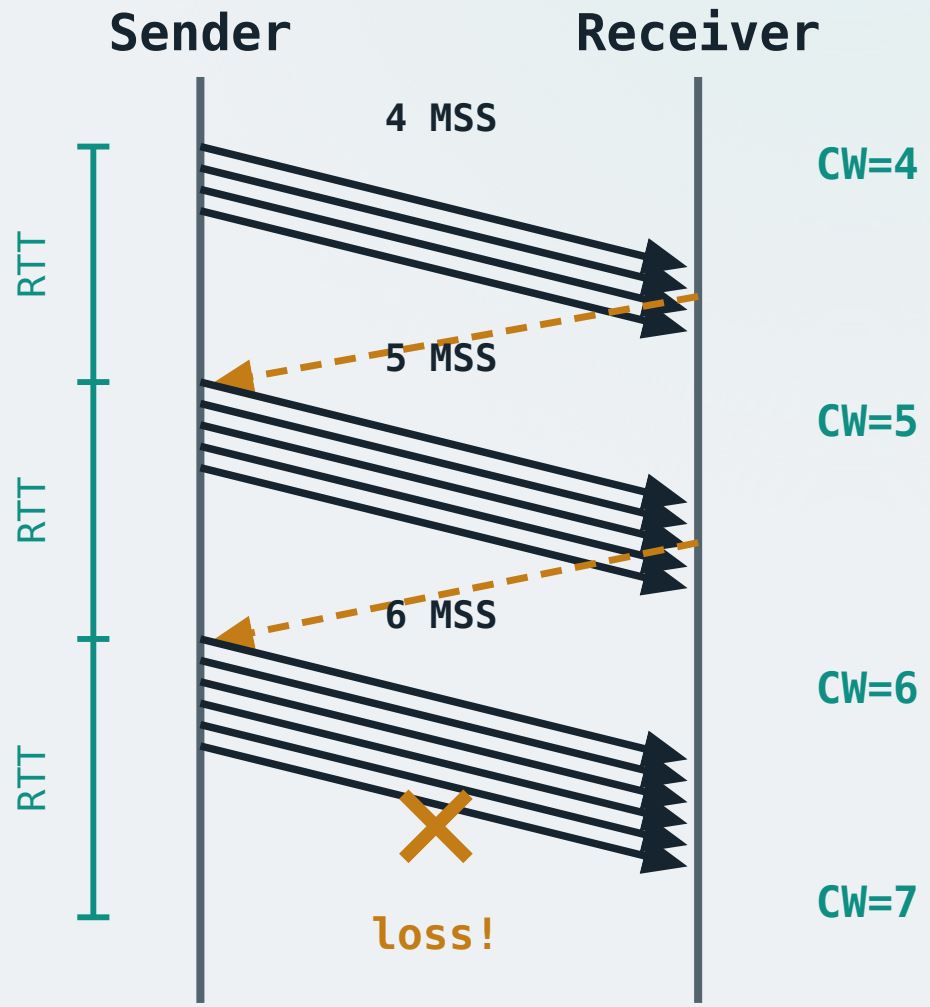
```
ADDITIVE INCREASE
If no loss detected: CongWin += 1 MSS per RTT
```

Multiplicative decrease

We keep additive increase until we hit a loss event (next couple of frames), where we cut CongWin in half.

```
MULTIPLICATIVE DECREASE
Upon loss event: CongWin = CongWin / 2
```

Together, this is known as the **AIMD** rule — Additive Increase, Multiplicative Decrease.



Why AIMD?

Multiplicative Increase, Additive Decrease (MIAD)

Multiplicative increase overshoots capacity aggressively, while additive decrease backs off too slowly — congestion persists.

Additive Increase, Additive Decrease (AIAD)

AIAD controls congestion correctly for a single flow, but it fails to address a different issue: **fairness**. The goal of fairness is that every flow sharing a bottleneck link should converge to an equal share of bandwidth.

> Suppose two flows share a 10 Mbps link: Flow A = 8 Mbps, Flow B = 2 Mbps. This is not fair — a good congestion control design should correct the imbalance while also preventing congestion.

> Additive increase: both add 1 Mbps → A = 9 Mbps, B = 3 Mbps. Total exceeds capacity → loss.

> Additive decrease: both subtract 1 Mbps → A = 8 Mbps, B = 2 Mbps. Back to the start — the unfairness never corrects.

AIAD does not solve the fairness problem.

Additive Increase, Multiplicative Decrease (AIMD)

Additive increase probes gently; multiplicative decrease reacts sharply. Converges to both **efficiency** and **fairness**.

Take-home question: using the same example (A = 8 Mbps, B = 2 Mbps), why does multiplicative decrease close the gap where additive decrease could not?

TCP · reacting to loss

Cutting CongWin in half is not the whole story. What TCP really adjusts on loss is the **slow-start threshold** — our empirical estimate of the network's capacity.

```
ON ANY LOSS EVENT
ssthresh = CongWin / 2
```

Case 1: Timeout

Set ssthresh = CongWin / 2.

Reset **CongWin = 1 MSS** and enter **slow start**.

> A timeout is a very serious signal of severe congestion — be conservative and restart from scratch.

Case 2: Three duplicate ACKs

Set ssthresh = CongWin / 2, then enter **fast recovery** (next frame).

This is the TCP Reno design. In TCP Tahoe, there is no distinction — both cases reset CongWin = 1 MSS and enter slow start.

TCP · fast recovery

Suppose CongWin = 6. Segments 1–3 have been acknowledged, and the sender has more data to transmit.

The sender fills the congestion window: segments 4–9 are now in flight.

The sender then receives three dup ACKs for segment 4. What does this mean?

Segment 4 is likely **lost** during transmission. But importantly, three ACKs means three of the in-flight packets 5–9 reached the receiver — suppose those are segments 5, 6, 8.

TCP · fast recovery

What should the sender do now?

Fast retransmit: immediately resend segment 4.

The sender enters **fast recovery**.

TCP · fast recovery

FAST RECOVERY — ENTER

`ssthresh = CongWin / 2 = 3`

`CongWin = ssthresh + 3 = 6`

Why inflate by 3?

- CongWin limits how many segments can be in flight. If we simply apply AIMD and set CongWin = ssthresh = 6/2 = 3, the window starts at base 4 and covers segments 4, 5, 6.
- However, by observing 3 dup ACKs, we know 3 segments among 5–9 are no longer in flight. Segments **5, 6, 8** are confirmed received and do not occupy in-flight slots.
- The only truly in-flight segments are **4, 7, 9**. This justifies inflating CongWin to 6 = ssthresh (3) + confirmed received (3), reflecting that only 3 segments are truly in flight.

TCP · fast recovery

FAST RECOVERY — PER DUP ACK

For each additional dup ACK: CongWin += 1 MSS

For the same reason, each additional dup ACK signifies one less segment is truly in flight — the window can grow by 1.

TCP · fast recovery

FAST RECOVERY — PER DUP ACK

For each additional dup ACK: CongWin += 1 MSS

For the same reason, each additional dup ACK signifies one less segment is truly in flight — the window can grow by 1.

Segment 7 received, yet segment 4 still pending (another dup ACK for 4): **CongWin = 7** — can now send segment 10.

TCP · fast recovery

FAST RECOVERY — PER DUP ACK

For each additional dup ACK: CongWin += 1 MSS

For the same reason, each additional dup ACK signifies one less segment is truly in flight — the window can grow by 1.

Segment 7 received, yet segment 4 still pending (another dup ACK for 4): **CongWin = 7** — can now send segment 10.

Segment 9 received, yet segment 4 still pending (another dup ACK for 4): **CongWin = 8** — can now send segment 11.

TCP · fast recovery exit

> Now suppose the retransmitted segment 4 has been received, so the receiver sends a cumulative ACK for everything through segment 9 — it sends ACK 10.

TCP · fast recovery exit

Now suppose the retransmitted segment 4 has been received, so the receiver sends a cumulative ACK for everything through segment 9 — it sends ACK 10.

FAST RECOVERY — EXIT RULE

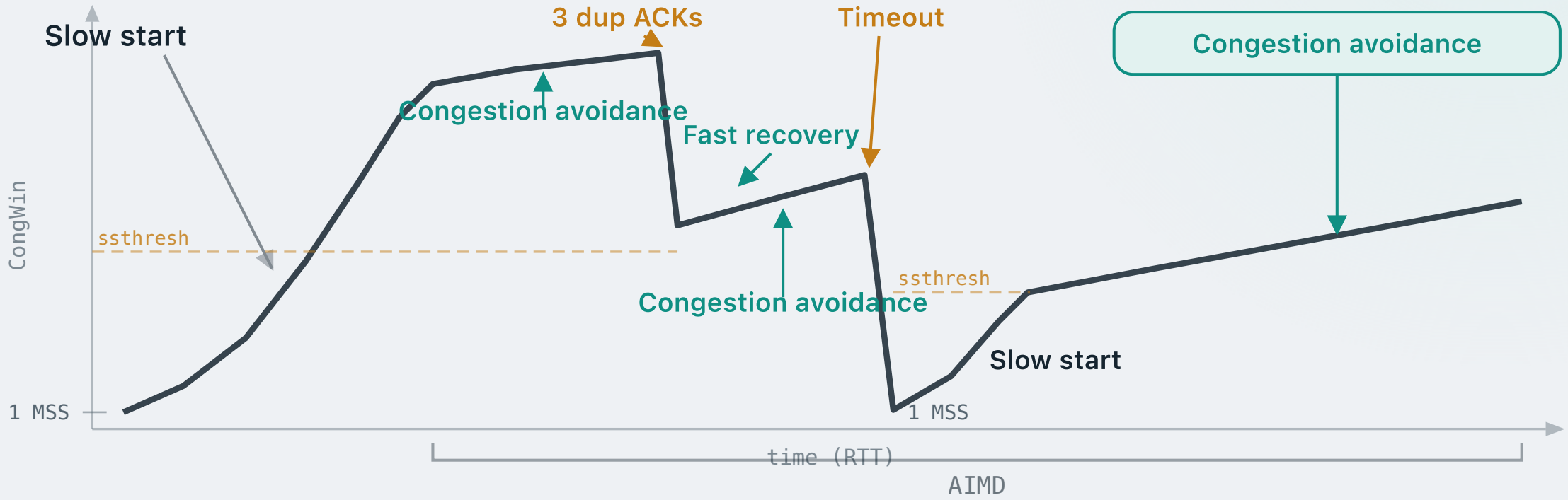
On new ACK:

- **Set CongWin = ssthresh**
- **Enter congestion avoidance**

The sender receives ACK 10. Base advances to segment 10. **CongWin = ssthresh = 3** — the inflated window deflates back to the halved value.

Enter **congestion avoidance**. Fast recovery is complete.

TCP · congestion control overview



Congestion control — summary

TCP begins in **slow start**, doubling CongWin each RTT until ssthresh, then grows linearly in **congestion avoidance**.

Three dup ACKs trigger **fast recovery** — ssthresh and CongWin halve. A **timeout** drops CongWin to 1 MSS and restarts slow start.

The resulting **AIMD sawtooth** continuously probes for available bandwidth while backing off sharply on congestion signals.

Acknowledgements

Lecture notes are developed based on slides from the following sources:

- Dr Mengmeng Ge's lecture notes for COSC264, University of Canterbury, 2024
- Assoc Prof Dan Kim's lecture notes for COSC264, University of Canterbury, 2017
- Prof Aleksandar Kuzmanovic's lecture notes for CS340, Northwestern University, 2005 — `https://users.cs.northwestern.edu/~akuzma/classes/CS340-w05/lecture_notes.htm`
- Dr Barry Wu's lecture notes for COSC264, University of Canterbury

References

James F. Kurose & Keith W. Ross, *Computer Networking: A Top-Down Approach Featuring the Internet*, Addison Wesley, 3rd edition, 2005.

Chapter 3 Transport Layer (3.1 – 3.3, 3.5 – 3.7)

William Stallings, *Data and Computer Communications*, Pearson, 10th edition, 2013.

Chapter 15 Transport Protocols

Andrew S. Tanenbaum & David J. Wetherall, *Computer Networks*, Prentice Hall, 5th edition, 2010.

Chapter 6 The Transport Layer (6.2, 6.3, 6.5)